SMARTQ

SMART VIRTUAL QUEUE & TOKEN MANAGER

A PROJECT REPORT

Submitted by

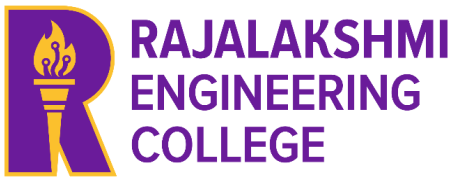
HEMAPRIYA B
(241401028)

in partial fulfilment of the End Semester Examination for the course

CB23F35 –
LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF TECHNOLOGY
COMPUTER SCIENCE AND BUSINESS SYSTEMS



RAJALAKSHMI ENGINEERING COLLEGE
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "SmartQ – Smart Virtual Queue & Token Manager" is the bonafide work of HEMAPRIYA B (241401028), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. K. R. Sowmia, Ph.D.,
HEAD OF THE DEPARTMENT
Associate Professor & Head
Department of Computer Science and Business Systems
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

M. L. Alphin Ezhil Manuel,
COURSE FACULTY
Assistant Professor (SG)
Department of Computer Science and Business Systems
Rajalakshmi Engineering College
Thandalam – 602 105

Submitted for the End Semester Examination (Individual Project) held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, Hemapriya B (241401028), hereby declare that the project report titled “SmartQ – Smart Virtual Queue & Token Manager”, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai
Date:

SIGNATURE OF THE STUDENT
Hemapriya B
241401028

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman **Mr. S. Meganathan, B.E., F.I.E.,**Chairperson **Dr. Thangam Meganathan, Ph.D.,** and Vice-Chairman **Mr. Abhay Shankar Meganathan, B.E., M.S.,** for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, **Dr. S. N. Murugesan, M.E., Ph.D.,** Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank **Dr. K. R. Sowmia, Ph.D.,** Associate Professor & Head, Department of Computer Science and Business Systems, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, **M. L. Alphin Ezhil Manuel,** Assistant Professor (SG), Department of Computer Science and Business Systems, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

HEMAPRIYA B

ABSTRACT

Service counters in clinics, college offices and banks are commonly managed with paper slips, registers and physical lines. Visitors cannot tell how long they will wait, and staff cannot see how many people are waiting at each counter or how long each service normally takes. The result is crowding, repeated enquiries and unevenly used staff time, and there is no record from which service performance can be assessed.

SmartQ – Smart Virtual Queue & Token Manager is a low-code application built on Zoho Creator to replace this manual practice with digital tokens. Low-code suits the problem because queue management is a data-entry and workflow task that the platform's forms, reports and dashboard components cover directly. The application (named Queue Management System in the Creator console) contains three linked forms – Service Counter, Token and Feedback – joined by lookup fields. An auto-number field and a token code (for example SQ-0015) identify each token, form workflows record the booking date and issue time, and Deluge scripts calculate the queue position and the estimated waiting time and recalculate it when a counter's average service time changes. Tokens move through the statuses Waiting, Serving, Served and No-show. Nine form workflows also send each customer an e-mail with the estimated wait and post alerts to the Zoho Cliq channel #smartq-notifications. Staff and administrators work with a Kanban board of service types, token reports including a pivot report, a colour-coded feedback list and a dashboard with status tiles, feedback counts and two charts. Access is governed by a CEO/Administrator and Service Counter Staff role hierarchy, three permission profiles (CEO/Administrator, Service Counter Staff and Customer), four data sharing rules and a customer portal.

The application was tested through its live interface, and the delivery of the customer e-mail and of the Cliq message was confirmed in the recipient mailbox and in the Cliq channel. Version 2.0 was published from the Development environment to Production on 31 October 2026. The report also records the defects found in the first build, the improvements made, and the limitations that remain, which are listed as future enhancements.

Keywords: Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Queue Management; Role-Based Access Control

PROJECT AT A GLANCE

Attribute	Details
Project Title	SmartQ – Smart Virtual Queue & Token Manager
Application Domain	Queue management and service counters (clinics, college offices, banks)
Student Name & Register No.	HEMAPRIYA B – 241401028
Zoho Creator Edition	Trial edition
Application Name (Link Name)	SmartQ
Published / Live URL	https://smartq-qms.zohocreatorportal.in (customer portal)
Demo Credentials (per role)	hemapriya.b.2024.csbs@gmail.com (admin)
Number of Forms / Subforms	3 (Service Counter, Token, Feedback) / 1 (Location & Operating Hours on Service Counter)
Number of Reports (by type)	9 reports and boards: All Service Counters, Service Types (Kanban), Token Report, Statuses, Time, upcoming event, Pivot Token Report, All Feedback (list), Feedback Board (list)
Number of Pages / Dashboards	1 (SmartQ Dashboard with token and feedback tiles and two charts)
Number of Workflows	Form: 9 (8 on Token, 1 on Service Counter) · Schedule, Approval, Blueprint, Batch: none
Deluge Functions / Custom Actions	Deluge scripts in the form workflows (waiting-time calculation and recalculation, Cliq posting) and the function Notify_Queue_Staff
Integrations	Zoho Cliq (connection zoho_cliq_connection; channel #smartq-notifications); Creator e-mail notifications to customers and staff
Roles & Profiles Defined	Roles: CEO/Administrator, Service Counter Staff · Permission profiles: CEO/Administrator, Service Counter Staff, Customer · Data sharing rules: 4
Environments Used	Development and Production
Date of Production Deployment	31 October 2026, 19:24 IST (version 2.0)

TABLE OF CONTENTS

PAGE

ABSTRACT v
PROJECT AT A GLANCE vi
LIST OF TABLES ix
LIST OF FIGURES x
LIST OF ABBREVIATIONS xi

INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 3
1.6 Organisation of the Report 3

LITERATURE SURVEY AND SYSTEM STUDY 4
2.1 Study of the Existing System 4
2.2 Limitations of the Existing System 4
2.3 Low-Code versus Traditional Development 5
2.4 Literature Survey 6
2.5 Emerging Trends in Low-Code Development 6
2.6 Proposed System 7

REQUIREMENT ANALYSIS 8
3.1 Stakeholders and User Roles 8
3.2 Functional Requirements 8
3.3 Non-Functional Requirements 10
3.4 Hardware and Software Requirements 10
3.5 Use Case Model 11

SYSTEM DESIGN 14
4.1 System Architecture 14
4.2 Data Model 15
4.3 Form Design 17
4.4 Process and Workflow Design 18
4.5 User Interface and Navigation Design 21
4.6 Access Control Design 23

TABLE OF CONTENTS

PAGE

IMPLEMENTATION 24
5.1 Application Setup 24
5.2 Forms and Data Capture 25
5.3 Reports 28
5.4 Pages and Dashboards 30
5.5 Deluge Scripting 31
5.6 Workflows and Automation 33
5.7 Integrations 34
5.8 User Management and Role-Based Access 37
5.9 AI and Analytics Features 38
5.10 Pro-Code Extension 38
5.11 Mobile and Portal Access 39

TESTING AND DEBUGGING 40
6.1 Testing Strategy 40
6.2 Test Cases 40
6.3 Debugging Deluge Scripts 43
6.4 Defect Log 45
6.5 User Acceptance Testing 46

DEPLOYMENT, SECURITY AND GOVERNANCE 47
7.1 Environments and Deployment Strategy 47
7.2 Performance Optimisation 48
7.3 Security Features 48
7.4 Governance and Administration 50

RESULTS AND DISCUSSION 51
8.1 Application Output 51
8.2 Achievement of Objectives 52
8.3 Comparison with the Existing System 52
8.4 Limitations of the Low-Code Approach 53
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 53

CONCLUSION AND FUTURE ENHANCEMENTS 54
9.1 Conclusion 54
9.2 Future Enhancements 54
9.3 Learning Outcomes and Career Relevance 55

REFERENCES 55

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 56

APPENDIX B – COURSE OUTCOME MAPPING 58

APPENDIX C – COMPLETE DELUGE SOURCE CODE 59

APPENDIX D – USER MANUAL 62

LIST OF TABLES

PAGE

Table 2.1: Comparison of Low-Code and Traditional Development for the SmartQ Application 5
Table 2.2: Summary of Literature Survey 6
Table 3.1: Stakeholders and User Roles 8
Table 3.2: Functional Requirements 8
Table 3.3: Non-Functional Requirements 10
Table 3.4: Hardware and Software Requirements 10
Table 3.5: Use Case Description – Issue Token (UC-03) 12
Table 3.6: Use Case Description – Update Token Status (UC-04) 12
Table 3.7: Use Case Description – Submit Feedback (UC-07) 13
Table 4.1: Entities and Relationships 15
Table 4.2: Field Specification – Service Counter Form 17
Table 4.3: Field Specification – Token Form 17
Table 4.4: Field Specification – Feedback Form 18
Table 4.5: Token Status Values and Transitions 19
Table 4.6: Permission Profiles and Their Design Intent 23
Table 5.1: Application Set-up Summary 24
Table 5.2: Forms Implemented 25
Table 5.3: Reports Implemented 29
Table 5.4: Components of the SmartQ Dashboard 31
Table 5.5: Deluge Scripts Implemented 31
Table 5.6: Workflows Implemented 33
Table 5.7: Integrations Implemented 35
Table 5.8: Data Sharing Rules 37
Table 6.1: Test Cases 41
Table 6.2: Summary of Test Results 43
Table 6.3: Debugging Observations 44
Table 6.4: Defect Log 45
Table 6.5: User Acceptance Feedback 46
Table 7.1: Deployment Checklist 47
Table 7.2: Performance Optimisation Opportunities (Proposed, Not Measured) 48
Table 7.3: Security Controls 49
Table 8.1: Mapping of Objectives to Outcomes 52
Table 8.2: Existing System versus Proposed Application 52
Table A.1: Feature Coverage Checklist 56
Table B.1: Mapping of Course Outcomes to Report Chapters 58

LIST OF FIGURES

PAGE

Figure 2.1: As-Is Process Flow of the Manual Queue at a Service Counter 4
Figure 3.1: Use Case Diagram of the SmartQ Application 11
Figure 4.1: System Architecture of the SmartQ Application 15
Figure 4.2: Entity–Relationship Diagram of SmartQ (* marks a mandatory field) 16
Figure 4.3: Token Processing Workflow (Lifecycle of a Token) 19
Figure 4.4: Form Events, Workflows and the Actions They Produce 20
Figure 4.5: Flowchart of the Recalculate Waiting Time Workflow 21
Figure 4.6: Navigation Structure of the SmartQ Application 22
Figure 5.1: Zoho Creator Notification “Publish to Production Completed” for Version 2.0 (e-mail address masked) 25
Figure 5.2: Service Counter Form in Live Mode 26
Figure 5.3: Token Form in the Form Builder with the Service Counter Lookup Properties . . 27
Figure 5.4: Token Form in Live Mode 27
Figure 5.5: Feedback Form in Live Mode 28
Figure 5.6: All Feedback Report 29
Figure 5.7: Service Types Kanban Report Grouped by Service Type 29
Figure 5.8: Record View of a Token Opened from the Time Report 30
Figure 5.9: Pivot Token Report. in the Report Designer 30
Figure 5.10: SmartQ Dashboard Page 31
Figure 5.11: Form Workflows of the Token Form 34
Figure 5.12: Form Workflow of the Service Counter Form 34
Figure 5.13: Zoho Cliq Connection Details (zoho_cliq_connection) 35
Figure 5.14: #smartq-notifications Channel in Zoho Cliq (member name masked) 35
Figure 5.15: Participants of the #smartq-notifications Channel (personal details masked) . . . 36
Figure 5.16: Customer E-mail “Your Token - Estimated Wait Time” 36
Figure 5.17: “New entry added” E-mail Notification for a Token (e-mail address masked) 36
Figure 5.18: Role Hierarchy (CEO/Administrator and Service Counter Staff) 37
Figure 5.19: Permission Profiles of SmartQ 37
Figure 5.20: Data Sharing Rules 38
Figure 5.21: Users of the Account (e-mail addresses and names masked) 38
Figure 5.22: Portal Settings and Sign-In Page of the SmartQ Portal 39
Figure 6.1: Deluge Script of the Recalculate Waiting Time Workflow 44
Figure 6.2: Deluge Script of the Customer Assigned Workflow 44

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
ER	Entity–Relationship
FR	Functional Requirement
IST	Indian Standard Time
NFR	Non-Functional Requirement
PII	Personally Identifiable Information
QMS	Queue Management System
RBAC	Role-Based Access Control
SMS	Short Message Service
UAT	User Acceptance Testing
UC	Use Case
UI	User Interface
URL	Uniform Resource Locator
Zia	Zoho’s AI assistant

CHAPTER 1

INTRODUCTION

1.1 Background

Clinics, college administrative offices, banks and similar service counters usually serve visitors on a first-come, first-served basis. A visitor arrives, joins a physical line or writes a name in a register, and then waits without any dependable indication of when the turn will come. Counter staff, in turn, have no consolidated view of how many people are waiting at each counter or how long each type of service normally takes. The result is crowding, repeated enquiries at the counter and uneven use of staff time.

SmartQ – Smart Virtual Queue & Token Manager is a low-code application built on Zoho Creator to replace this manual practice with digital tokens. Each customer is issued a numbered token (for example SQ-0015) for a chosen service counter. The application records the booking date and issue time, works out the customer's queue position and estimated waiting time, and e-mails that estimate to the customer. Counter staff track every token through the states Waiting, Serving, Served and No-show, receive alerts in a Zoho Cliq channel, and see the whole queue on a dashboard. After service, customers can record a rating and comments in a Feedback form, and the administrator can follow each feedback item until it is resolved.

The application was first built as two linked forms with a dashboard and was reviewed against its own test evidence. The findings of that review shaped the present version, which adds a Feedback form, automatic e-mail and Cliq notifications, recalculation of waiting times, validation of queue position, a customer portal, role-specific permission profiles and a richer dashboard. Version 2.0 of the application, named *Queue Management System* in the Creator console and branded *SmartQ* on its dashboard, was published to the Production environment on 31 October 2026 (Figure 5.10).

1.2 Problem Statement

Manual queue handling at service counters gives neither visitors nor staff reliable information about waiting time or queue length. A low-cost digital solution is required that issues sequential tokens, records the time of issue, estimates the wait for each token, tracks the service status of every token, informs the customer and the staff automatically, collects feedback and presents the state of the queue on a single dashboard. The main pain points are:

- Visitors cannot see their position in the queue or estimate how long they must wait.

- Staff cannot see, at a glance, how many tokens are waiting, being served or completed at each counter.
- Paper slips and registers are easily lost or mis-sequenced, and they produce no usable statistics about waiting and service.
- Neither side is alerted when a token is issued or assigned, and complaints about service are not recorded or followed up.

1.3 Objectives

The project set out to achieve the following objectives:

- To design a data model of linked forms – Service Counter, Token and Feedback – with lookup relationships from each token to its service counter and from each feedback item to its token.
- To automate token numbering, recording of booking date and issue time, queue position and estimated waiting time using a Creator auto-number field, form workflows and Deluge scripting, and to recalculate waiting times when a counter's average service time changes.
- To track every token through the statuses Waiting, Serving, Served and No-show, with validation of the queue position on submission.
- To notify customers by e-mail with their estimated waiting time and to alert staff through a Zoho Cliq channel when a token is generated or assigned.
- To provide reports and a dashboard that show tokens by status, service types as a Kanban board, a pivot report of tokens, feedback ratings and resolution status, and live counts and charts.
- To capture customer feedback with a rating, comments and a resolution status so that service quality can be followed up.
- To control access through a role hierarchy, three permission profiles (CEO/Administrator, Service Counter Staff and Customer), data sharing rules and a customer portal.

1.4 Scope of the Project

In scope. The implemented application contains three forms (Service Counter, Token and Feedback) with a Location & Operating Hours sub-section on the Service Counter form, a set of reports and Kanban boards for service types, tokens and feedback, one dashboard page, nine form workflows (eight on the Token form and one on the Service Counter form), Deluge scripts for waiting-time calculation and Zoho Cliq posting, e-mail notifications, a Zoho Cliq

connection, a role hierarchy of CEO/Administrator and Service Counter Staff, three permission profiles, four data sharing rules and a customer portal. The application is described as a general-purpose queue and token manager for a service-counter environment and is demonstrated with sample counters such as Express Lane, Mobile Unit, Night Shift, Specialist Support and Main Service Desk.

Out of scope. The following were considered but are not part of the delivered application and are treated in this report as future enhancements: SMS notification to customers, a native mobile application configuration, priority tokens, multi-branch operation, automatic marking of No-show tokens by a schedule, and integration with third-party services through REST API calls.

1.5 Justification for a Low-Code Approach

A queue manager is a data-entry and workflow application: a few forms, a handful of reports, simple calculations and notifications. These are exactly the building blocks that a low-code application platform provides as configurable components. On Zoho Creator the forms, their lookup relationships, the reports and Kanban boards and the dashboard page were assembled with the form builder, report builder and page builder, and only the waiting-time calculation and the notification messages required scripting in Deluge. Hosting, authentication, user management, the customer portal, responsive rendering and record storage are supplied by the platform, so the project effort could be concentrated on the queue logic rather than on infrastructure. Published studies of low-code practice report that drag-and-drop development and out-of-the-box components speed up development and that low-code is most favoured for process- and workflow-centred applications, although practitioners also report limits in customisation [4], [5].

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2

LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The existing arrangement at most service counters is manual. A visitor arrives and either takes a printed slip, writes a name in a register or simply joins a physical line. The visitor then waits with no visibility of the queue position. A staff member calls a name or number aloud; if the person is absent the entry is skipped or called again later, and after service the register is updated by hand. Figure 2.1 summarises this commonly followed practice. The description is a general account of manual queue handling and is not a survey of one named institution.

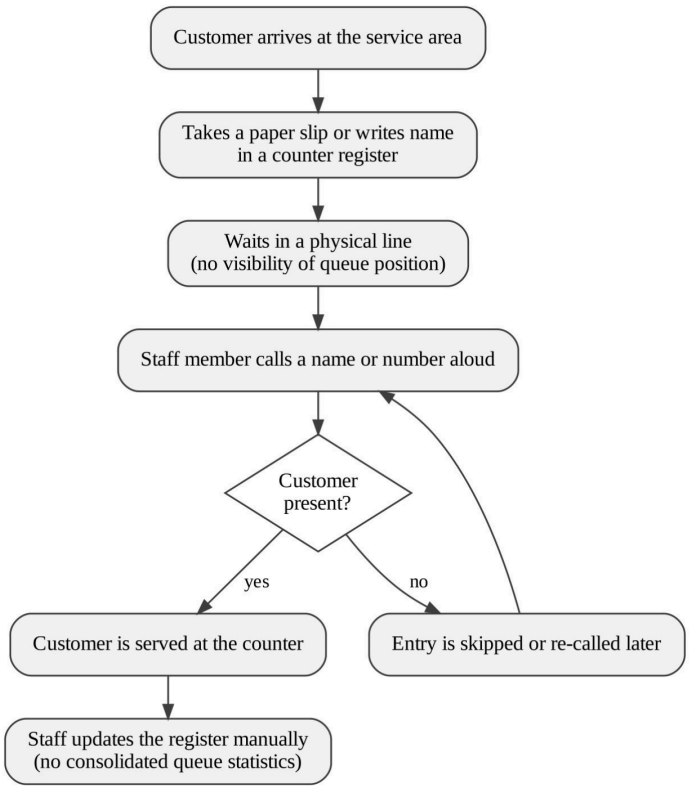


Figure 2.1: As-Is Process Flow of the Manual Queue at a Service Counter

2.2 Limitations of the Existing System

- **No visibility of waiting time.** Visitors cannot tell how long they will wait, which leads to repeated enquiries and crowding near the counter.
- **No consolidated queue view.** Staff cannot see the number of waiting or served visitors per counter without counting entries in a register.
- **Unreliable sequencing.** Paper slips and handwritten registers are prone to loss, duplication and out-of-order calling.

- **No service statistics.** Average service time, number of no-shows and counter load are not captured, so staffing decisions are made without data.
- **No automated alerts.** Neither the customer nor the staff are notified when the queue changes; they must watch the line themselves.
- **No feedback loop.** Complaints and compliments are given informally at the counter and are not recorded, rated or followed up.

2.3 Low-Code versus Traditional Development

Table 2.1 compares a traditional pro-code implementation of the queue manager with the low-code implementation on Zoho Creator. The comparison is qualitative; no development-time or cost measurements were recorded during the project, so none are claimed.

Table 2.1: Comparison of Low-Code and Traditional Development for the SmartQ Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Database design, back-end services, user interface and deployment pipeline must each be built and integrated.	Forms, reports and the dashboard are assembled in builders; Deluge was needed only for waiting-time, validation and notification logic.
Skills required	Programming in a front-end and a back-end language, SQL, hosting and security administration.	Platform familiarity and basic Deluge scripting.
Hosting and infrastructure	Servers or cloud services must be provisioned, secured and maintained.	Hosted and maintained by the platform.
Customer access and mobile	A customer-facing web site and a separate mobile client or responsive front end must be developed.	A customer portal is enabled from the Portal settings, and the platform renders applications on desktop, tablet and phone previews.
Notifications and integration	E-mail servers, chat APIs, authentication tokens and error handling must be coded.	E-mail actions are configured in workflows, and the Zoho Cliq connection is created in the console and called from one Deluge statement.
Security and access control	Authentication, sessions, role logic and record sharing must be implemented in code.	Users, roles, permission profiles and data sharing rules are configured in Settings.
Maintenance and change	Changes require code changes, testing and redeployment.	Fields, reports and workflows are changed in the builders; scripts are edited in the Deluge editor and published to Production.
Customisation limits	Practically unlimited.	Bounded by platform components and Deluge capabilities (see Section 8.4).
Cost	Developer time plus infrastructure and licences.	Platform subscription; the project was built on a trial edition as shown in the application header.

2.4 Literature Survey

Table 2.2 summarises the sources reviewed. Two are empirical studies of low-code practice; three are the vendor documentation sets for the platform services on which the application depends. Only findings that could be confirmed from the cited pages are recorded.

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	Y. Luo, P. Liang, C. Wang, M. Shahin and J. Zhan, 2021 [4]	Characteristics and Challenges of Low-Code Development: The Practitioners' Perspective	Analysis of Stack Overflow and Reddit posts found that drag-and-drop interfaces and out-of-the-box components make low-code platforms easy to learn and speed up development, that they are favoured for automated processes and workflows, and that practitioners hold conflicting views of their advantages and disadvantages.	Supports the choice of a low-code platform for a workflow-oriented queue application and motivates the limitations discussion in Section 8.4.
2	M. A. Al Alamin, S. Malakar, G. Uddin, S. Afroz, T. B. Haider and A. Iqbal, 2021 [5]	An Empirical Study of Developer Discussions on Low-Code Software Development Challenges	Analysis of Stack Overflow posts on nine low-code platforms identified 13 discussion topics in four categories; customisation was the largest category, and dynamic event handling was both the most popular and the hardest topic to obtain accepted answers for.	Highlights event handling and customisation as risk areas, which corresponds to the form-event workflows and Deluge scripts used in SmartQ.
3	Zoho Corporation [1]	Zoho Creator Resource Center	Central documentation for building applications, using Deluge, managing users and assigning roles and permissions.	Reference for forms, reports, pages, workflows, portals and user permissions.
4	Zoho Corporation [2]	Introduction to Deluge (Zoho Deluge Help)	Describes Deluge as Zoho's scripting language used to manipulate data, for example by creating or updating records.	Reference for the waiting-time scripts and the Cliq notification script.
5	Zoho Corporation [3]	Cliq Help Center	Documentation for Zoho Cliq, the team chat service, including its integrations with other applications.	Reference for the Cliq channel used for staff notifications.

2.5 Emerging Trends in Low-Code Development

Three general trends are relevant to this project. First, low-code platforms increasingly embed AI assistance in their builders; the Zia assistant icon is visible in the Zoho Creator builder screens captured for this project, although no AI feature was used in SmartQ. Second, platforms emphasise integration with the surrounding service ecosystem, which SmartQ exercises through e-mail actions and a Zoho Cliq connection. Third, as more business users build applications, governance concerns such as role design, data sharing, data privacy

settings and environment management become more important; SmartQ addresses these with a role hierarchy, permission profiles, sharing rules and a Development-to-Production release. The empirical literature also records that customisation limits and event handling remain recurring difficulties for developers [4], [5]. These trends are stated qualitatively and are not backed by market figures.

2.6 Proposed System

The proposed system is the SmartQ application. Service counters are configured once with a name, service type, status, capacity, operating hours, manager details and average service time. A token is issued by choosing a counter on the Token form, either by staff or by the customer through the portal; the application numbers the token automatically, records the booking date and issue time, calculates the queue position and estimated waiting time, and e-mails the estimate to the customer. Tokens move through the states Waiting, Serving, Served and No-show and are visible in token reports, a status board, a pivot report and a dashboard that shows live counts and charts. Workflows post alerts to a Zoho Cliq channel so that staff are informed when a token is generated or assigned. Customers and staff can record feedback with a rating, and the administrator tracks its resolution. Access is controlled through a role hierarchy, permission profiles and data sharing rules. The proposed system therefore removes the manual register, gives staff a single view of every counter, gives each customer an estimated wait and gives the administrator a measurable record of service quality.

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

The roles below correspond to the roles and permission profiles defined in the application settings.

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
CEO / Administrator	Top role in the role hierarchy; permission profile "CEO/Administrator".	Complete administrative control: token management, service counter management, customer feedback, dashboards and all application records; manages users, roles and permissions.
Service Counter Staff	Role beneath the administrator in the hierarchy; permission profile "Service Counter Staff".	Operational access to the assigned counter only; issues tokens and updates token status; cannot delete records or modify counter settings.
Customer (portal user)	Token holder who signs in to the customer portal; permission profile "Customer".	Generates tokens, views the status of own tokens, accesses public counter information and submits feedback, without access to other customers' records or administrative features.
Zoho Cliq (system actor)	Team chat service connected to the application.	Receives the staff notifications posted to the #smartq-notifications channel.

3.2 Functional Requirements

Table 3.2 lists the functional requirements. FR-01 to FR-16 correspond to features implemented in the application; FR-17 and FR-18 are proposed requirements that have not been implemented.

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall allow an administrator to create service counters with Counter Name, Service Type, Counter Status and Average Service Time as mandatory fields, plus Max Capacity, location and operating hours, and manager details.	Admin	High
FR-02	The system shall group service counters by service type (Registration, Document Verification, Billing, Consultation and Other) and present them as a Kanban board.	Admin, Staff	Medium

FR ID	Requirement	Role(s)	Priority
FR-03	The system shall allow a token to be issued capturing customer name, e-mail, phone, address and the chosen service counter.	Staff, Customer	High
FR-04	The system shall generate sequential token numbers automatically (Auto Number field and a Token Number code of the form SQ-0015).	System	High
FR-05	The system shall record the booking date and the time of issue (Issued At) for each token and set the initial status automatically.	System	High
FR-06	The system shall calculate and store the queue position and the estimated waiting time of a new token from the tokens waiting at the same counter and the counter's average service time.	System	High
FR-07	The system shall recalculate the estimated waiting times of waiting tokens when a service counter record is edited.	System	Medium
FR-08	The system shall track each token in one of four states: Waiting, Serving, Served, No-show, and validate the queue position when a token is submitted or edited.	Staff	High
FR-09	The system shall e-mail the customer the estimated waiting time when a token is created.	System	High
FR-10	The system shall post a message to a Zoho Cliq channel when a token is generated and when a customer is assigned to a service counter.	System	Medium
FR-11	The system shall provide token reports (list, status board, time-based, upcoming-event and pivot views).	Admin, Staff	High
FR-12	The system shall provide a dashboard showing token counts by status, feedback counts by resolution status, an estimated-waiting-time chart, a service-type capacity chart and quick links.	Admin, Staff	Medium
FR-13	The system shall collect customer feedback linked to a token, with rating, comments, source, staff name, resolution status and follow-up details.	Customer, Staff	High
FR-14	The system shall list feedback with colour-coded ratings and present it as a Kanban board by resolution status.	Admin, Staff	Medium
FR-15	The system shall restrict access through a role hierarchy, three permission profiles and data sharing rules.	Admin	High
FR-16	The system shall provide a customer portal with sign-in so that customers can use the application without a Creator licence.	Customer	Medium
FR-17	(Proposed) The system shall remind the customer by SMS or e-mail when the token is near the front of the queue.	System	Proposed
FR-18	(Proposed) The system shall mark a Waiting token as No-show automatically after a defined time.	System	Proposed

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Reports and the dashboard shall open without perceptible delay for the record volumes of a single service area. No formal timing measurement was recorded.
NFR-02	Security	Users shall access only the forms, reports and pages permitted by their role and permission profile; customers shall not see other customers' records.
NFR-03	Usability	A token shall be issued with a single form submission; the application shall be usable in a standard web browser and through the customer portal.
NFR-04	Availability	The application shall rely on the availability of the Zoho Creator service; no separate hosting is maintained by the project.
NFR-05	Maintainability	Business logic shall be confined to named workflows and functions so that it can be reviewed and changed without redesigning the forms.
NFR-06	Auditability	Each token shall carry its booking date and issue time, and each service counter shall carry creation information (Created By and Created At).
NFR-07	Reliability of notifications	A customer e-mail and a staff alert shall be produced for every successfully submitted token; their delivery is checked in the mailbox and the Cliq channel.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator (trial edition indicated in the application header), with Development and Production environments
Scripting language	Deluge (Zoho's scripting language for Creator workflows and functions)
Browser	Google Chrome
Other Zoho services	Zoho Cliq (staff notification channel #smartq-notifications, reached through the connection zoho_cliq_connection); Creator e-mail notifications to customers and staff
Customer access	Customer portal published at https://smartq-qms.zohocreatorportal.in
Mobile	The application header offers desktop, tablet and phone preview toggles; a dedicated mobile configuration was not created
Client hardware	Any computer or tablet able to run a current web browser, with an internet connection

3.5 Use Case Model

Figure 3.1 shows the use cases of the application. The administrator manages service counters, roles, permissions and sharing, views the dashboard and reviews feedback; staff issue tokens, update token status, view the token reports and boards, and review feedback; customers use the portal to issue their own token, view its status and submit feedback. The administrator also performs the staff use cases, as indicated by the dashed generalisation arrow, which is consistent with the administrator sitting above staff in the role hierarchy. Issuing a token includes two system actions – the wait-time e-mail and the staff notification – whose secondary actors are the e-mail service and Zoho Cliq.

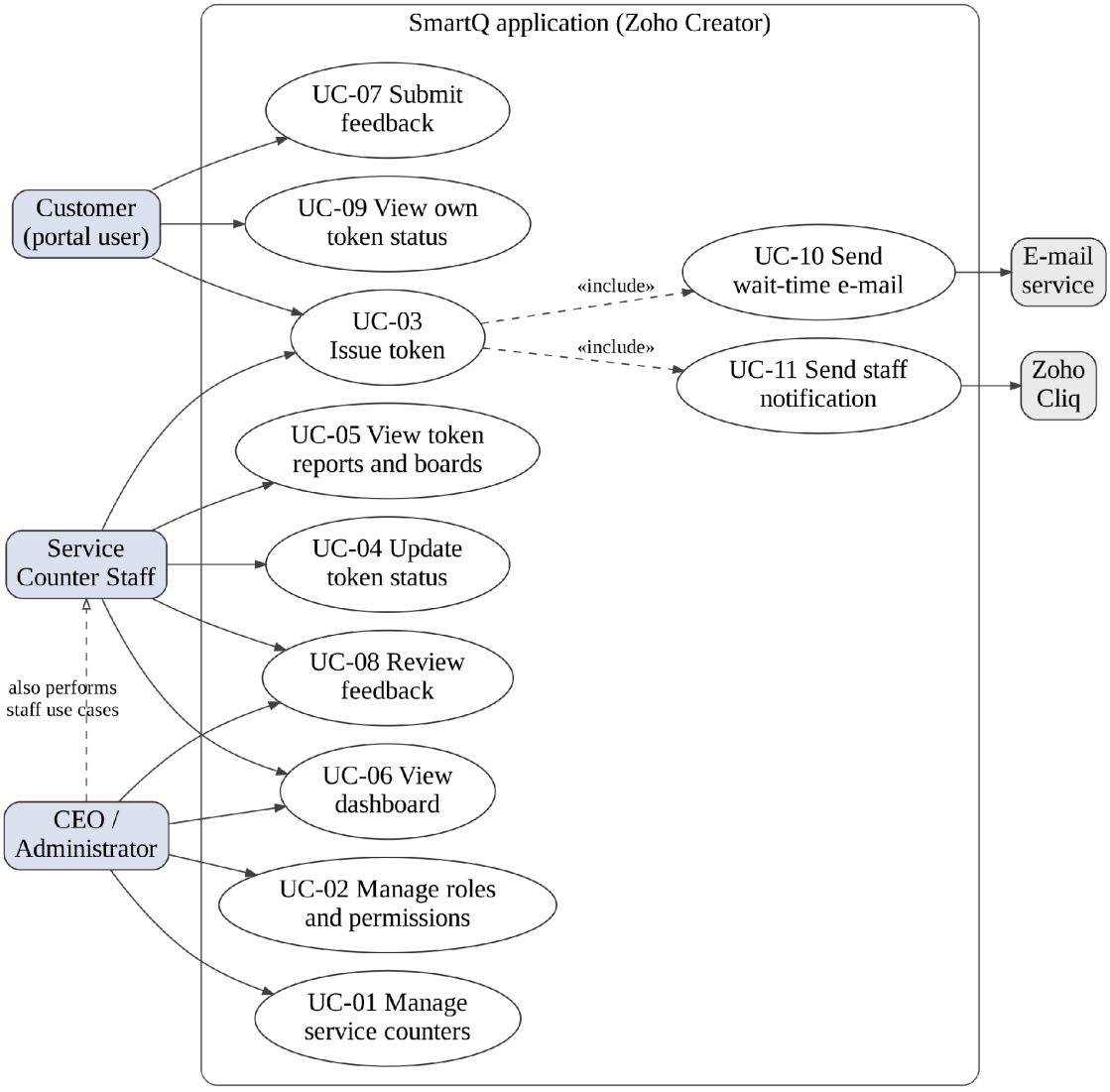


Figure 3.1: Use Case Diagram of the SmartQ ApplicationTables 3.5 to 3.7 describe three key use cases in detail.

Table 3.5: Use Case Description – Issue Token (UC-03)

Use Case ID	UC-03
Use Case Name	Issue Token
Actor(s)	Staff or Customer (primary); Administrator; E-mail service and Zoho Cliq (secondary)
Pre-condition	At least one Service Counter record exists; the actor has access to the Token form.
Main Flow	1. The actor opens the Token form from the Token menu section or the "Add New Token" quick link. 2. On load, the form pre-fills the Token Number (for example SQ-0015) and Issued At, and sets the initial status. 3. The actor enters the customer name, e-mail address, phone and address, and selects a Service Counter. 4. The actor clicks Submit. 5. The queue-position validation runs; the system assigns the next Token No. automatically. 6. The waiting-time workflow stores the Queue Position and Estimated Waiting Time. 7. The customer receives an e-mail with the estimated wait, and a "New token generated" message is posted to the #smartq-notifications channel.
Alternate Flow	If a mandatory field (Customer Name or Email Address) is empty, or the validation workflow rejects the entry, the form shows an error and no token is created.
Post-condition	A Token record with status Waiting appears in the token reports and on the status board, and the dashboard counts increase.
Related FR	FR-03, FR-04, FR-05, FR-06, FR-08, FR-09, FR-10

Table 3.6: Use Case Description – Update Token Status (UC-04)

Use Case ID	UC-04
Use Case Name	Update Token Status
Actor(s)	Staff (primary); Administrator; Zoho Cliq (secondary)
Pre-condition	A Token record exists in the Waiting or Serving state; the actor holds the Service Counter Staff or CEO/Administrator profile.
Main Flow	1. The actor opens the status board or the Token record. 2. The actor changes the status to Serving when the customer is called. 3. After service, the actor changes the status to Served, or to No-show if the customer is absent. 4. On successful submission of the edit, the Update Token Status and Customer Assigned workflows run, and an assignment message is posted to the Cliq channel.
Alternate Flow	A token may be moved to No-show directly from Waiting.
Post-condition	The token appears under its new status in the reports and board; the dashboard Token Status tiles reflect the change.
Related FR	FR-08, FR-10, FR-11, FR-12

Table 3.7: Use Case Description – Submit Feedback (UC-07)

Use Case ID	UC-07
Use Case Name	Submit Feedback
Actor(s)	Customer or Staff (primary); Administrator
Pre-condition	A Token record exists; the actor has access to the Feedback form.
Main Flow	1. The actor opens the Feedback form. 2. The actor selects the Token Number (mandatory), which fills the customer and counter details. 3. The actor selects a Rating, enters Comments, the Feedback Date (mandatory) and the Feedback Source. 4. Staff Name, Resolution Status and Follow-Up Details (including “Follow Up Required”) are completed for review. 5. The actor clicks Submit.
Alternate Flow	If the Token Number or Feedback Date is empty, the form shows a mandatory-field error.
Post-condition	The record appears in the All Feedback list with a colour-coded rating and in the feedback board, and the dashboard counts of Escalated, Resolved and Pending feedback are updated.
Related FR	FR-13, FR-14, FR-12

CHAPTER 4

SYSTEM DESIGN

This chapter describes how SmartQ is designed, before Chapter 5 shows how it was built. It covers the architecture, the data model, the form specifications, the token lifecycle and workflows, the navigation and dashboard design, and the access-control design. The design described here is that of the deployed application, the Queue Management System (version 2.0) on Zoho Creator, which was published to production on 31 October 2026.

4.1 System Architecture

SmartQ runs entirely inside Zoho Creator. It has no custom front end, no separate back end and no external database. Figure 4.1 shows the architecture. Service Counter Staff and the CEO / Administrator use the application through a web browser, and customers use the customer portal published at https://smartq-qms.zohocreatorportal.in, where they sign in with an e-mail address or sign up.

Presentation layer. Three forms (Service Counter, Token and Feedback), a set of list, Kanban and pivot reports, and the SmartQ Dashboard page.

Logic layer. Nine form workflows written with Deluge: eight on the Token form and one on the Service Counter form. They pre-fill values when a form opens, calculate waiting times, validate the queue position and send notifications.

Data layer. The records of the three forms, held by Creator and related by lookup fields: a Token refers to one Service Counter, and a Feedback record refers to one Token.

Notification channels. A message is posted to the Zoho Cliq channel #smartq-notifications through the connection zoho_cliq_connection, and e-mail is sent to the customer and to the application owner.

Access control. A two-level role hierarchy (CEO / Administrator above Service Counter Staff), three permission profiles and four data-sharing rules decide who can reach each component.

Three design decisions shaped this architecture. First, all business rules are kept in named, event-driven workflows, so that they can be reviewed and changed without redesigning the forms. Second, relationships between records are held in lookup fields rather than typed text, so that a token cannot point to a counter that does not exist. Third, notification is done with the platform's own Cliq connection and e-mail actions, so that no separate integration service has to be built or hosted.

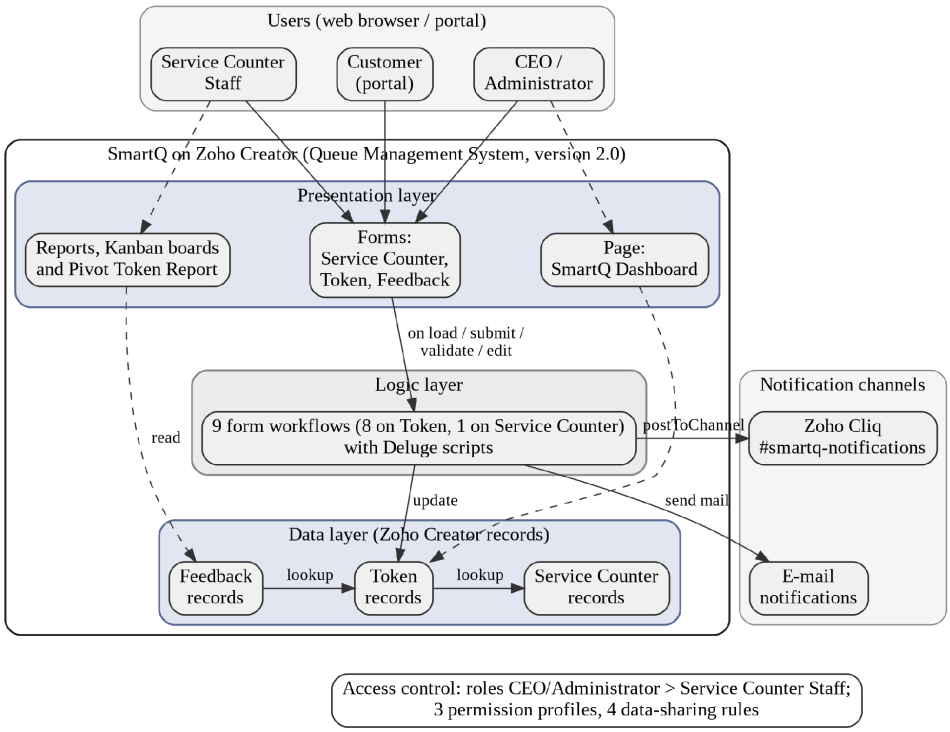


Figure 4.1: System Architecture of the SmartQ Application

4.2 Data Model

Zoho Creator stores data as records of forms. A form therefore plays the role that a table plays in a relational database, a field plays the role of a column, and a lookup field plays the role of a foreign key that points to a record of another form. SmartQ uses three forms. Service Counter holds the configuration of each counter and carries a subform, Location & Operating Hours, with one row per day. Token holds one record per issued token and carries a Service Counter lookup, so one counter can have many tokens and each token belongs to one counter. Feedback holds a customer's rating and comments and carries a Token Number lookup, so one token can have many feedback records. Figure 4.2 shows the entities and their relationships, and Table 4.1 summarises them.

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship	Implemented Using
Service Counter	Counter Name, Service Type, Counter Status, Max Capacity, Average Service Time, manager details, operating-hours subform	Token	One-to-Many	Service Counter lookup in Token
Location & Operating Hours (subform)	Location, Day, Operating Time, Closing Time, Is Closed	Service Counter	Many-to-One	Subform rows inside the counter record

Entity (Form)	Key Fields	Related To	Relationship	Implemented Using
Token	Token No., Token Number, Booking Date, Issued At, Customer Name, Phone, Email, Address, Service Counter, Estimated Waiting Time, Queue Position, Token Status	Service Counter, Feedback	Many-to-One (counter); One-to-Many (feedback)	Service Counter lookup; target of the Feedback lookup
Feedback	Token Number, Customer Name, Service Counter, Rating, Comments, Feedback Date, Feedback Source, Staff Name, Resolution Status, Follow Up Required	Token	Many-to-One	Token Number lookup

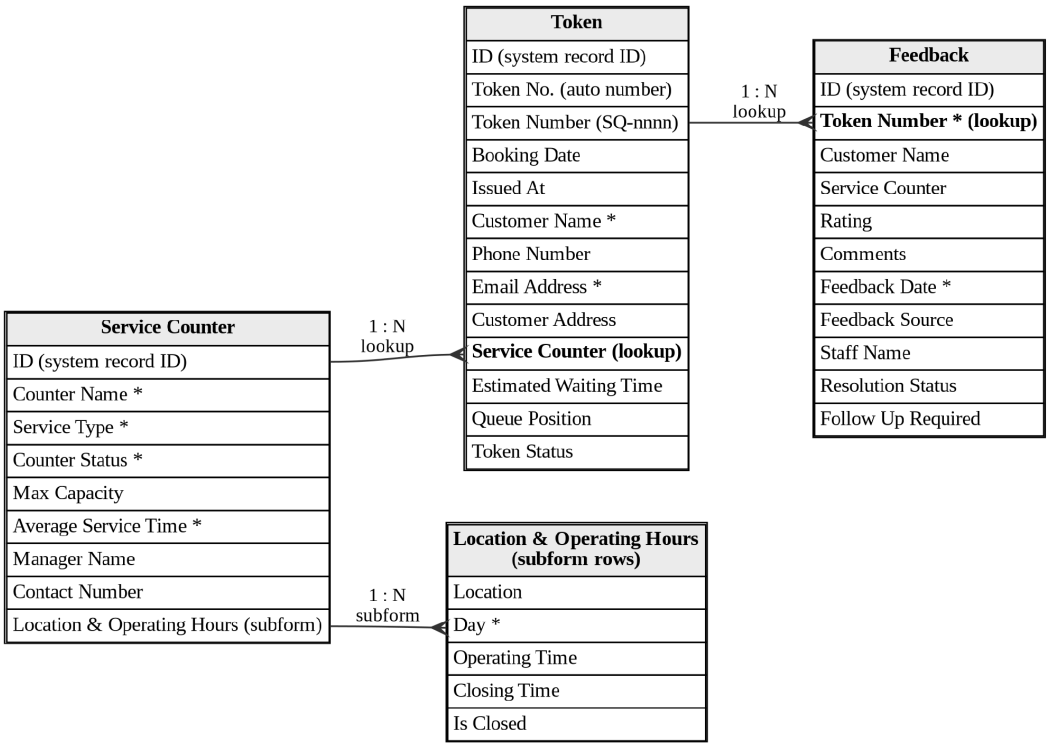


Figure 4.2: Entity–Relationship Diagram of SmartQ (* marks a mandatory field)

Data consistency is supported in five ways.

- **Lookups.** The counter on a token and the token on a feedback record are chosen from existing records, so mistyped counter names or token numbers cannot enter the data.
- **Automatic numbering.** Token No. is an Auto Number field, and the Token Number shown to the customer follows the pattern SQ-nnnn (for example SQ-0015 on the open form), so each token has a unique, increasing identifier.
- **Fixed value sets.** Counter Status (Closed, Open, Other) is a radio field, and Service Type and Token Status are drop-downs, so only the defined values can be stored.

- **Mandatory fields.** Counter Name, Service Type, Counter Status, Average Service Time and the operating Day on Service Counter; Customer Name and Email Address on Token; Token Number and Feedback Date on Feedback.
- **Validation on submission.** The Validate Queue Position workflow runs when a Token is created or edited, before the record is saved (Section 4.4).

4.3 Form Design

Tables 4.2 to 4.4 specify the fields of the three forms. Each form is arranged in titled sections so that the person entering data sees related fields together. Field names, sections, mandatory flags and field types were read from the form builder and the live forms. Where scripts refer to a field, its link name is given in the Remarks column.

Table 4.2: Field Specification – Service Counter Form

Field Label	Field Type	Mandatory	Section and Remarks
Counter Name	Single Line	Yes	Counter Information
Service Type	Drop Down	Yes	Counter Information; values include Registration, Document Verification, Billing, Consultation and Other
Counter Status	Radio	Yes	Counter Information; Closed, Open, Other
Max Capacity	Number	No	Counter Information; used by the dashboard chart Service Type by Max Capacity
Location	Single Line	No	Subform Location & Operating Hours
Day	Drop Down	Yes	Subform Location & Operating Hours
Operating Time	Time	No	Subform; format HH:mm:ss
Closing Time	Time	No	Subform; format HH:mm:ss
Is Closed	Drop Down	No	Subform; marks a day on which the counter is closed
Manager Name	Name	No	Counter Manager Details
Contact Number	Phone	No	Counter Manager Details; country-code selector (+91)
Average Service Time	Number	Yes	Service Configuration; minutes per customer; link name average_service_time

Table 4.3: Field Specification – Token Form

Field Label	Field Type	Mandatory	Section and Remarks
Token No.	Auto Number	System	Token Information; sequential record number
Token Number	Single Line	No	Token Information; pre-filled as SQ-nnnn when the form opens; link name Token_Number

Field Label	Field Type	Mandatory	Section and Remarks
Booking Date	Date	No	Token Information
Issued At	Date-Time	No	Token Information; pre-filled with the date and time when the form opens
Customer Name	Single Line	Yes	Customer Details; link name customer_name
Phone Number	Phone	No	Customer Details; country-code selector (+91)
Email Address	Email	Yes	Customer Details; used to send the waiting-time e-mail
Customer Address	Address	No	Customer Details
Service Counter	Lookup (Service Counter)	No	Service Allocation; displays Counter Name as a drop-down; link name service_counter
Estimated Waiting Time	Number	No	Queue & Waiting Information; minutes; link name estimated_waiting_time
Queue Position	Number	No	Queue & Waiting Information
Token Status	Drop Down	No	Queue & Waiting Information; Waiting, Serving, Served, No-show; link name token_status

Table 4.4: Field Specification – Feedback Form

Field Label	Field Type	Mandatory	Section and Remarks
Token Number	Lookup (Token)	Yes	Token & Service Details; links the feedback to a token
Customer Name	Single Line	No	Token & Service Details
Service Counter	Lookup (Service Counter)	No	Token & Service Details
Rating	Drop Down	No	Customer Feedback; rating values 2, 3 and 4 appear in the records; shown in colour in the All Feedback report
Comments	Multi Line	No	Customer Feedback
Feedback Date	Date	Yes	Customer Feedback
Feedback Source	Drop Down	No	Customer Feedback; values seen: Phone, Online, Email
Staff Name	Name	No	Staff & Service Review
Resolution Status	Drop Down	No	Staff & Service Review; values seen: Pending, Escalated; Resolved is counted on the dashboard
Follow Up Required	Drop Down	No	Staff & Service Review

4.4 Process and Workflow Design

Token lifecycle. Figure 4.3 shows the lifecycle of a token. When the Token form opens, the Token Number, the Issued At time and the initial status Waiting are already filled. When the form is submitted, the queue position and the estimated waiting time are stored and the

notifications are sent. All later changes of status are manual edits by staff: Waiting to Serving when the token is called, Serving to Served when service ends, and Waiting or Serving to No-show when the customer is absent. Table 4.5 lists the four status values. The design contains no scheduled or automatic status change.

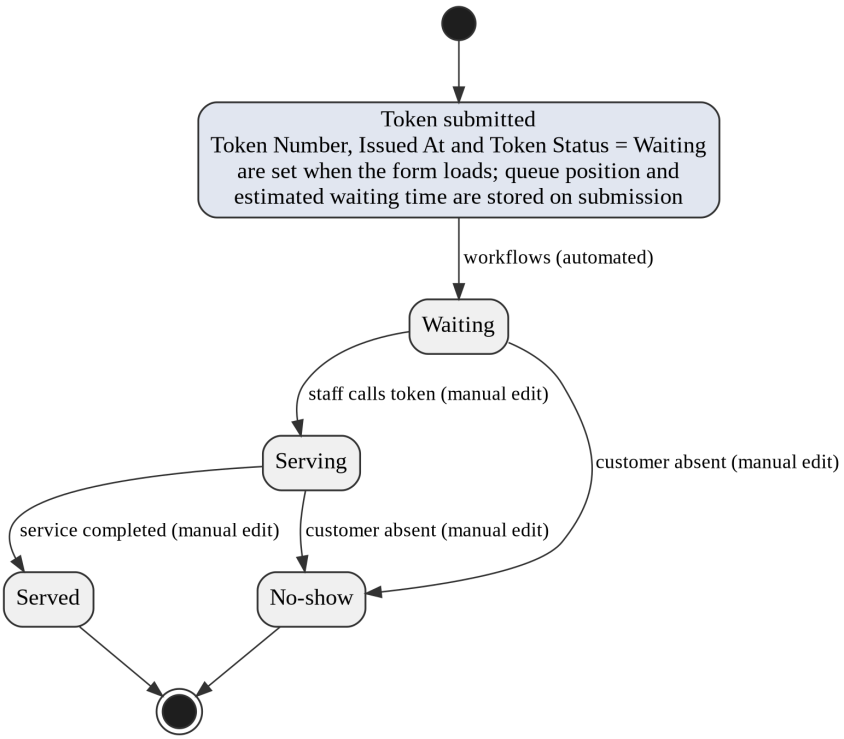


Figure 4.3: Token Processing Workflow (Lifecycle of a Token)

Table 4.5: Token Status Values and Transitions

Status	Entered When	Changed By	Where It Appears
Waiting	The token is submitted; the status is pre-set to Waiting when the form opens	Form default (automated)	Statuses report; dashboard tile Waiting
Serving	Staff call the token to the counter	Staff (manual edit)	Dashboard tile Serving
Served	Service for the token is complete	Staff (manual edit)	Dashboard tile Served
No-show	The customer does not respond when called	Staff (manual edit)	Dashboard tile No-show

Workflows. The nine form workflows, all enabled, are listed in Table 5.6, and Figure 4.4 shows which form event starts each of them. Eight are defined on the Token form and one on the Service Counter form. The event decides when a workflow runs: Load of the form runs while the form is being opened, Successful form submission runs after the record is saved, and Validations on form submission runs before the record is saved.

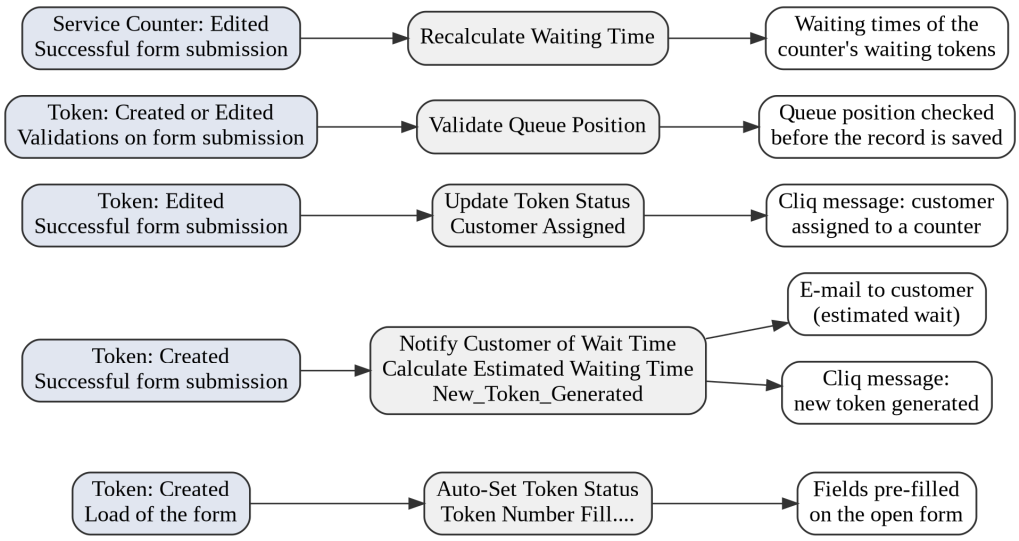


Figure 4.4: Form Events, Workflows and the Actions They Produce

Notification design. Three messages leave the application. When a token is created, the customer receives an e-mail with the estimated waiting time (for example, "Your estimated waiting time is 95 minutes.") and staff see a Cliq message in #smartq-notifications. When a token is edited, a second Cliq message announces that a customer has been assigned to a counter. Both Cliq messages are sent through the connection zoho_cliq_connection. In addition, Creator's own application notification, "Queue Management System : New entry added", e-mails the submitted token data to the application owner.

Estimated waiting time. The estimate is held in the Estimated Waiting Time field of each token. It is stored when the token is created and refreshed by the Recalculate Waiting Time workflow, whose logic is shown in Figure 4.5. When a Service Counter record is edited and its average service time is present and greater than zero, the workflow fetches the tokens of that counter whose status is Waiting, numbers them from 1, and sets each token's estimated waiting time to

$$\textit{Estimated Waiting Time} = \textit{Average Service Time} \times \textit{Position}$$

where Position is the place of the token among the waiting tokens of that counter. For example, with an average service time of 5 minutes, the first waiting token is estimated at 5 minutes, the second at 10 and the third at 15.

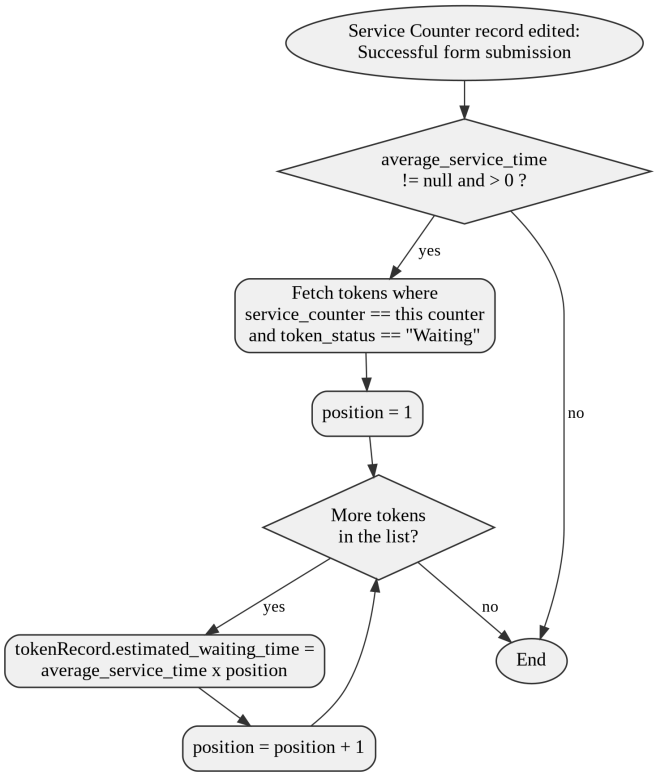


Figure 4.5: Flowchart of the Recalculate Waiting Time Workflow

The design has stated limitations. The recalculation runs when a counter record is edited, not each time a token is served, so the estimate is a snapshot between edits. It assumes that the average service time is constant, treats all tokens equally (there are no priority tokens) and takes no account of the time left on the token currently being served. Positions are assigned in the order in which the fetch returns the records. Recalculating whenever a token changes status is listed as a future enhancement in Chapter 9.

4.5 User Interface and Navigation Design

Figure 4.6 shows the navigation structure of the application. The side menu has the SmartQ Dashboard and three sections, each pairing a data-entry form with its reports. The Feedback section holds the Feedback form, the All Feedback report and the Feedback Board. The Service Counters section holds the Service Counter form, the All Service Counters report and the Service Types Kanban report. The Token section holds the Token form and the reports Token Report, Statuses, Time, upcoming event and two Pivot Token Report entries. App Preferences is a separate menu item.

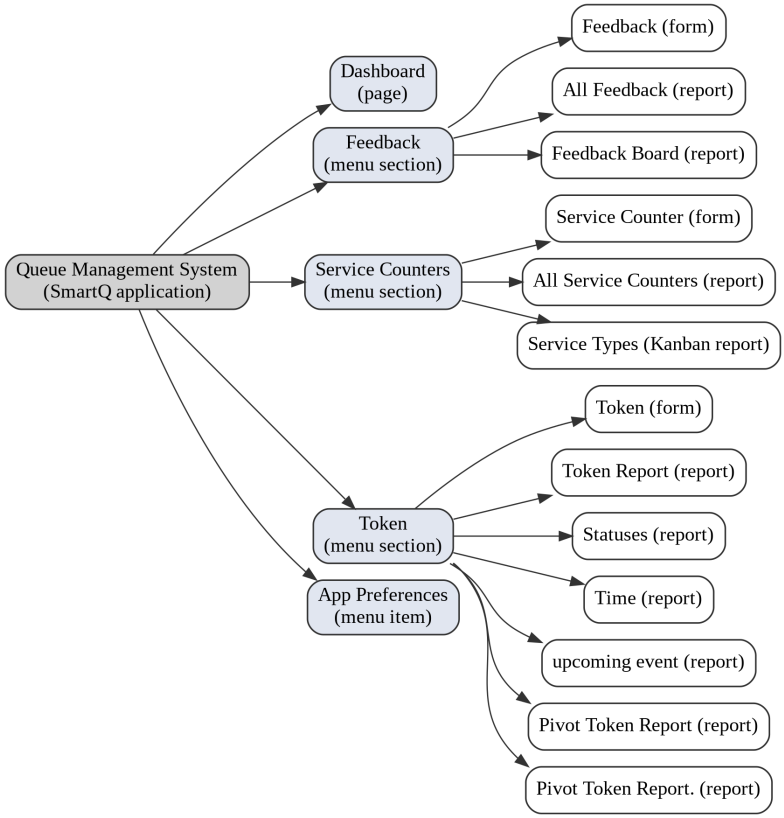


Figure 4.6: Navigation Structure of the SmartQ Application

Dashboard. The SmartQ Dashboard is the landing page for staff and administrators. Under the title "SmartQ – Smart Virtual Queue & Token Manager" it combines: summary tiles for feedback (Count of Escalated, Count of Resolved, Count of Pending and Total Feedback); a Token Status block with the tiles Serving, Served, No-show and Waiting; a Quick Links panel with Add New Feedback, Add New Service Counter and Add New Token; a scatter chart of Estimated Waiting Time by Queue Position; a bar chart of Service Type by Max Capacity; and the All Feedback list, in which the rating is shown in colour. A person can therefore see the state of the queue and the state of customer feedback on one screen and open a form in one click.

Forms and reports. Forms are divided into titled sections (for example Token Information, Customer Details, Service Allocation and Queue & Waiting Information on the Token form), so that a staff member fills the form from top to bottom. List reports show records in rows, the Service Types report shows service counters as cards grouped by service type, and the pivot reports summarise tokens in a table. The interface uses the platform's dark side menu with a purple accent and can be previewed in desktop, tablet and phone layouts.

4.6 Access Control Design

Access is designed on four levels. Staff and administrators sign in with their Zoho accounts, and customers sign in to the portal with their e-mail address. A role hierarchy places the CEO / Administrator above Service Counter Staff. Permission profiles define what each kind of user may do with each form, report and page. Data-sharing rules extend record visibility between roles.

Permission profiles. Three profiles are defined: CEO/Administrator, Customer and Service Counter Staff. Table 4.6 records the intent of each profile as written in its description. How each profile is set against the individual forms, reports and pages is shown in Chapter 5.

Table 4.6: Permission Profiles and Their Design Intent

Profile	Intended Access	Intended Restriction
CEO/Administrator	Complete administrative control over SmartQ: token management, service counter management, customer feedback, reports, dashboards and all application records	None
Customer	Generate tokens, view their own token status, access public counter information and submit feedback	No access to other customers' records or to administrative features
Service Counter Staff	Operational access to the assigned counter; may update token status	Cannot delete records or modify counter settings

Role hierarchy and data sharing. In Zoho Creator a role hierarchy governs data visibility between users, whereas a permission profile governs what a user may do. SmartQ has two roles, CEO/Administrator at the top and Service Counter Staff beneath it. Four data-sharing rules, all read-only, make records visible across the two roles, as shown in Table 5.8. Together they let the administrator see the tokens and feedback handled by staff, and let staff see the service counters that the administrator configures. Two of the rules share Token records in the same direction and could be merged into one.

Customer access. Customers reach the application through the portal, whose sign-in page asks for an e-mail address and offers sign-up and password recovery. The Customer profile limits a signed-in customer to generating tokens, tracking their own tokens and submitting feedback, which keeps the personal details of other customers private.

CHAPTER 5

IMPLEMENTATION

5.1 Application Setup

The application was created in a Zoho Creator account under the name Queue Management System. Its dashboard carries the project title SmartQ – Smart Virtual Queue & Token Manager, and its tile on the portal shows the initials "QMS". The components are organised in the application menu as the Dashboard page, three sections (Feedback, Service Counters and Token) and the App Preferences item (Figure 4.6). The three sections hold the three forms of the application (Feedback, Service Counter and Token) together with their reports, which are listed in Table 5.3. Nine form workflows are attached to the forms: eight on Token and one on Service Counter (Section 5.6).

The application was developed in the Development environment (the builder header offers the button "Access Development Live"), and version 2.0 was published to Production on 31 October 2026 at 19:24 IST. Figure 5.1 shows the notification e-mail that Zoho Creator sent after the publish. The application is offered to customers through a public portal at https://smartq-qms.zohocreatorportal.in (Section 5.11). The Creator header shows that the account was on a trial edition with five days remaining when the screenshots were taken. Table 5.1 summarises the set-up. **Table 5.1:** Application Set-up Summary

Attribute	Details
Application name	Queue Management System (SmartQ)
Application link name	SmartQ
Production version	2.0, published on 31 Oct 2026 at 19:24 IST (Figure 5.1)
Portal URL	https://smartq-qms.zohocreatorportal.in (portal type: Public; Figure 5.22)
Forms	3: Service Counter, Token, Feedback (Section 5.2)
Reports	10 menu entries (Section 5.3)
Pages	1: Dashboard (Section 5.4)
Workflows	9 form workflows: Token 8, Service Counter 1, Feedback 0 (Section 5.6)
Integrations	Zoho Cliq (connection zoho_cliq_connection); e-mail notifications (Section 5.7)
Roles and profiles	2 roles (CEO/Administrator, Service Counter Staff); 3 permission profiles; 4 data-sharing rules (Section 5.8)
Account users	3 users, one of them the Super Admin; 0 portal users (Figure 5.21, Figure 5.22)
Edition	Trial edition.

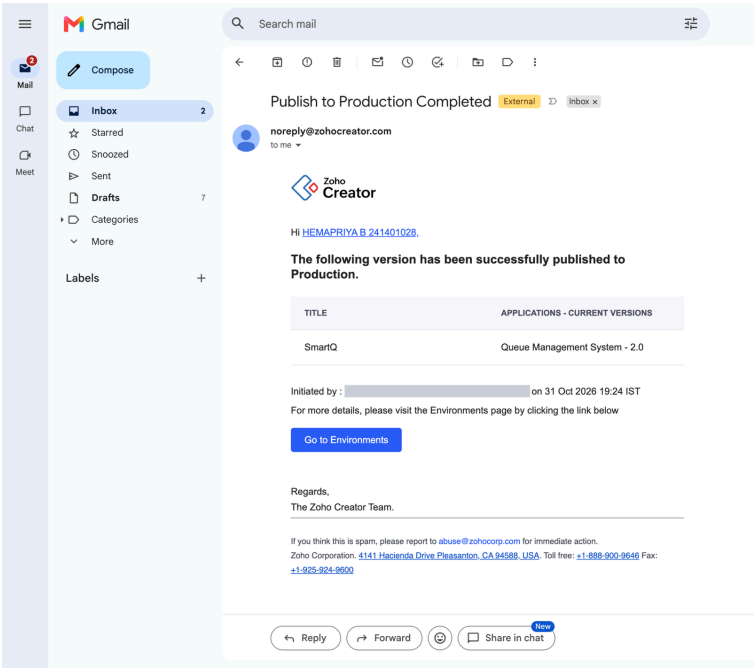


Figure 5.1: Zoho Creator Notification "Publish to Production Completed" for Version 2.0 (e-mail address masked)

5.2 Forms and Data Capture

SmartQ stores its data in three forms: Service Counter, Token and Feedback. Each form is divided into titled sections, so the live form reads as a guided entry sheet. Table 5.2 summarises the forms; the full field specification is given in Tables 4.2 to 4.4 of Chapter 4.

Table 5.2: Forms Implemented

Form	Sections	Mandatory Fields	Special Features
Service Counter	Counter Information; Location & Operating Hours; Counter Manager Details; Service Configuration	Counter Name, Service Type, Counter Status, Average Service Time; Day (in the subform)	Subform for operating hours; radio field for status; phone field with country code; 1 workflow
Token	Token Information; Customer Details; Service Allocation; Queue & Waiting Information	Customer Name, Email Address	Auto-number Token No.; lookup to Service Counter; Token Number and Issued At filled on load; 8 workflows
Feedback	Token & Service Details; Customer Feedback; Staff & Service Review; Follow-Up Details	Token Number, Feedback Date	Lookup to Token; check box for follow-up; no workflow

Service Counter form. Figure 5.2 shows the form in live mode. The Counter Information section holds Counter Name, Service Type (drop-down), Counter Status (radio buttons: Closed, Open, Other) and Max Capacity; Counter Name, Service Type and Counter Status are mandatory. The Location & Operating Hours section is a subform: each row records a Location, a Day (mandatory), an Operating Time, a Closing Time and an Is Closed flag, and the Add New link adds a row, so one counter can carry different hours for different days. Counter Manager Details holds the manager's name and a contact number with a

country-code selector (+91 shown). Service Configuration holds Average Service Time, which is mandatory and drives the waiting-time calculation (Section 5.5).

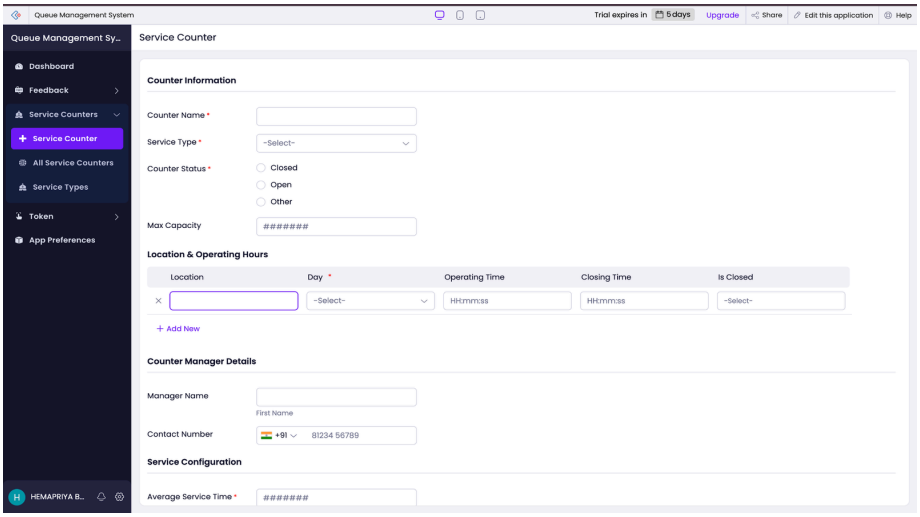


Figure 5.2: Service Counter Form in Live Mode

Token form. Figure 5.3 shows the Token form in the builder and Figure 5.4 shows it in live mode. Token Information holds Token No., Token Number, Booking Date and Issued At. Token No. is an auto-number field. Token Number is a text field that is already filled with the next number in the form SQ-nnnn (SQ-0015 in Figure 5.4), and Issued At already holds the current date and time when the form opens; both are the work of workflows that run on the load of the form (Section 5.6). Customer Details holds Customer Name and Email Address, which are mandatory, and Phone Number (with a +91 country-code selector) and Customer Address. Service Allocation holds the Service Counter lookup. Queue & Waiting Information holds Estimated Waiting Time and Queue Position; both are empty on the form and are filled by the application after submission. The token's status is held on the record (Figure 5.8). In the field properties of Figure 5.3 the lookup has the link name service_counter, shows Counter Name from the Service Counter form, is displayed as a drop-down and has Allow new entries selected. Mandatory is not selected for it, so a token can be submitted without a counter.The form collects personal data (customer name, phone number, e-mail address and postal address).

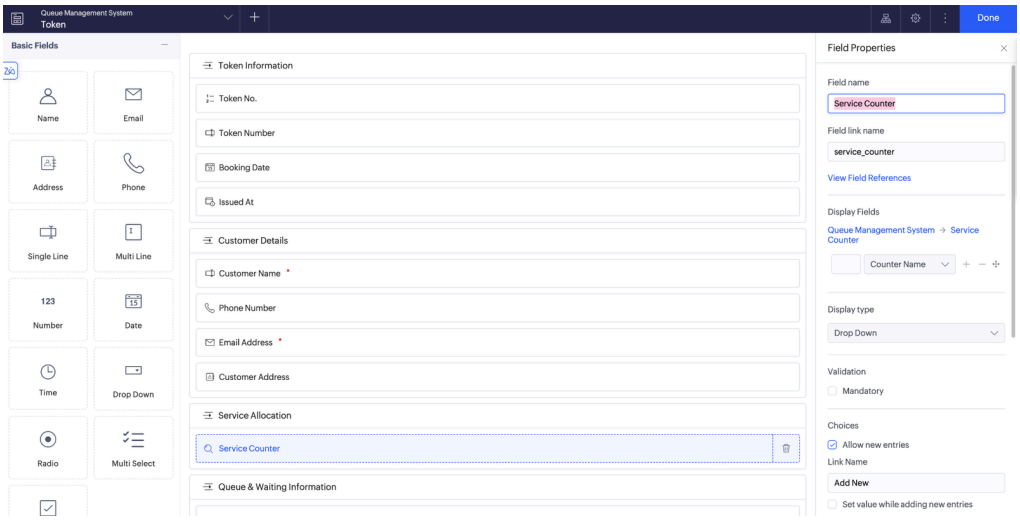


Figure 5.3: Token Form in the Form Builder with the Service Counter Lookup Properties

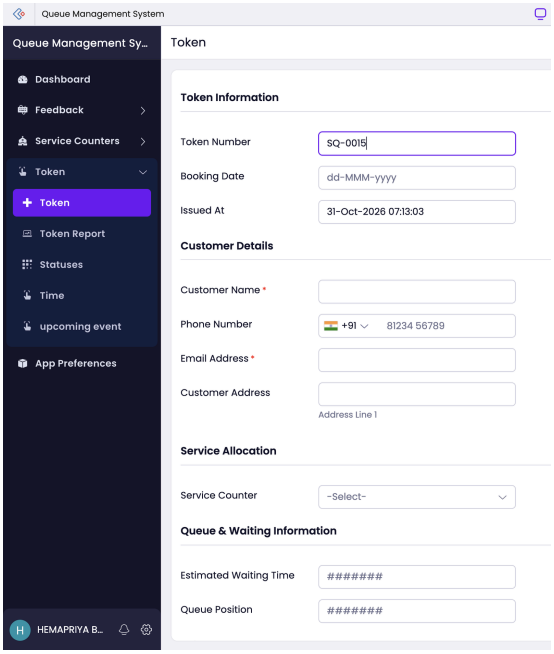


Figure 5.4: Token Form in Live Mode

Feedback form. Figure 5.5 shows the form. Token & Service Details holds a mandatory Token Number lookup, Customer Name and Service Counter. Customer Feedback holds Rating (drop-down), Comments, Feedback Date (mandatory) and Feedback Source (drop-down). Staff & Service Review holds Staff Name and Resolution Status, and Follow-Up Details holds a Follow Up Required check box. The Token Number lookup ties every feedback record to one token, which gives the one-to-many relationship shown in Figure 4.2. The form has no workflow attached (Figure 5.11).

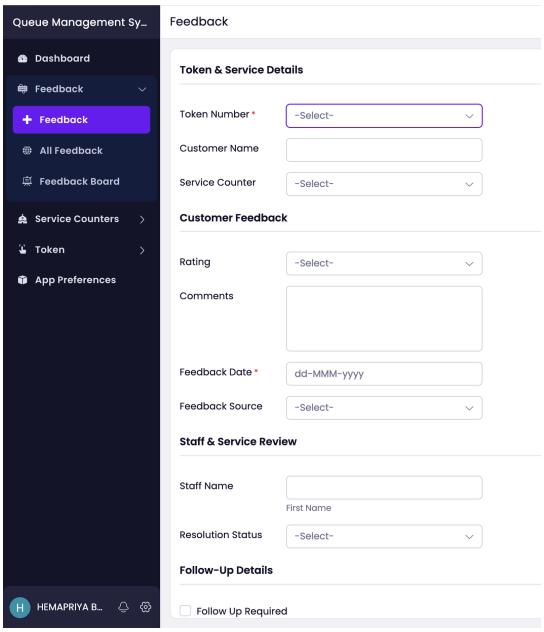


Figure 5.5: Feedback Form in Live Mode

5.3 Reports

The application menu lists ten report entries, which Table 5.3 summarises.

All Feedback (Figure 5.6) is a list report of the Feedback form with the columns Token Number, Customer Name, Service Counter, Rating, Comments, Feedback Date, Feedback Source, Staff Name and Resolution Status; the Rating cells are colour-coded. It holds five records, which agree with the dashboard (Section 5.4). **Service Types** (Figure 5.7) is a Kanban report on the Service Counter form that groups counters into columns by service type. The visible columns are Registration, Document Verification, Billing and Consultation, and each card shows a manager's name above the counter name (for example David Lee and Mobile Unit); the Document Verification and Consultation columns are empty.

Time is shown through its record view in Figure 5.8, which lists every field of one Token record: Service Counter, Email Address, Issued At, Booking Date, Token No., Phone Number, Customer Name, Token Status, Estimated Waiting Time, Customer Address, Queue Position, Token Number and Added Time. The view offers Edit, Duplicate and More actions and a comment link. The Token Number of this sample record is empty, whereas a new record receives its SQ-nnnn number when the form opens (Figure 5.4). **Pivot Token Report.** (the full stop is part of the name in the builder) is shown in the report designer in Figure 5.9. The Report Customization panel offers legend positions, but the preview area is empty and shows only the Open Report Builder button, so the configuration of this report is The output of Token Report, Statuses, upcoming event, Feedback Board, All Service Counters and Pivot Token Report is not shown in the supplied screenshots, so their types are marked for verification in Table 5.3.

Table 5.3: Reports Implemented

Report Name	Menu Section	Type	Source Form	Evidence / Purpose
All Feedback	Feedback	List	Feedback	Figure 5.6; five records; embedded on the dashboard
Feedback Board	Feedback	Kanban	Feedback	Output not shown; grouping by Resolution Status is assumed
All Service Counters	Service Counters	List	Service Counter	Output not shown; lists the counter records
Service Types	Service Counters	Kanban	Service Counter	Figure 5.7; columns by Service Type
Token Report	Token	List	Token	Output not shown
Statuses	Token	Kanban	Token	Output not shown; name suggests grouping by Token Status
Time	Token	Timeline	Token	Record view in Figure 5.8
upcoming event	Token	Pivot Table	Token	Output not shown
Pivot Token Report	Token	Pivot / chart	Token	Output not shown

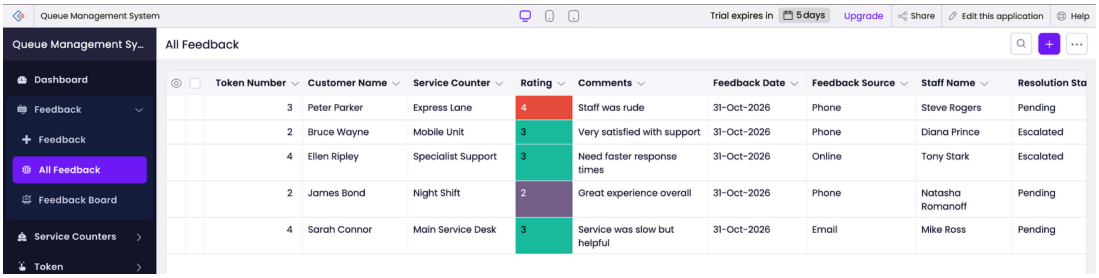


Figure 5.6: All Feedback Report

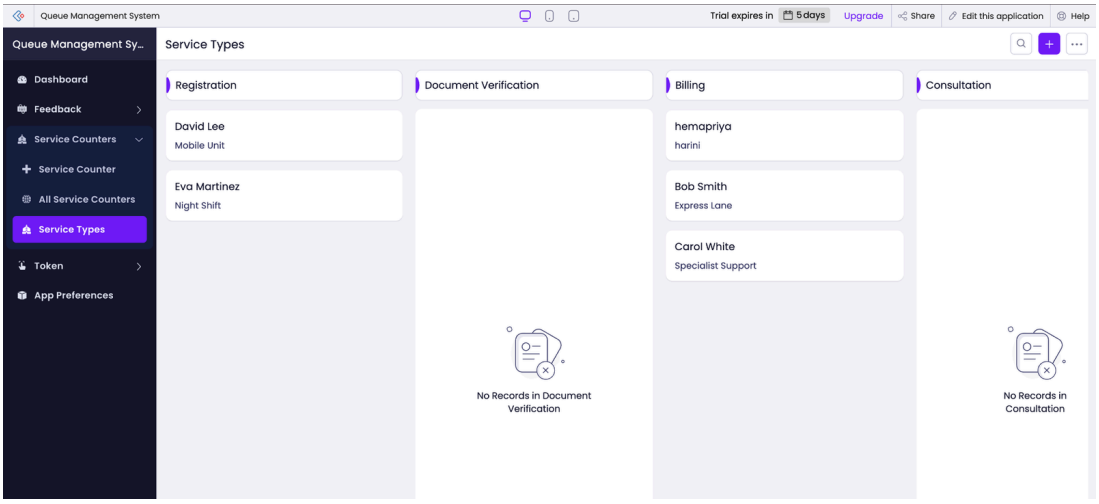


Figure 5.7: Service Types Kanban Report Grouped by Service Type

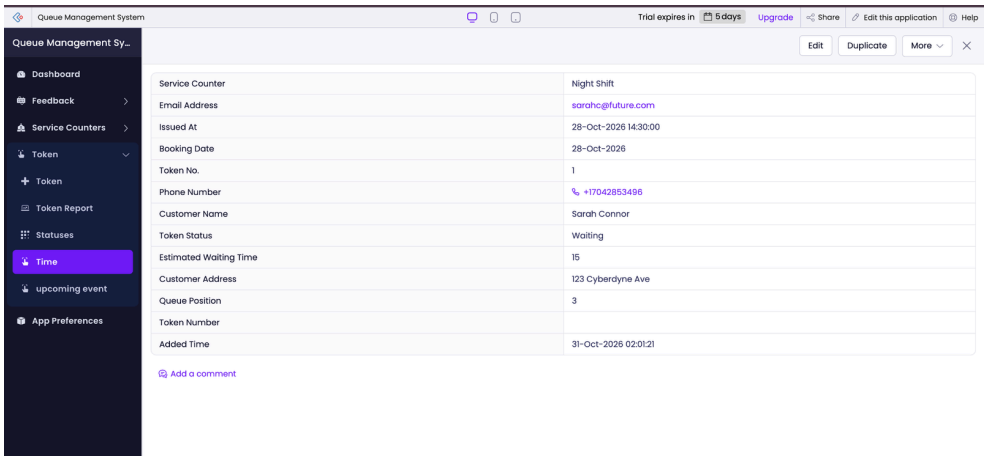


Figure 5.8: Record View of a Token Opened from the Time Report

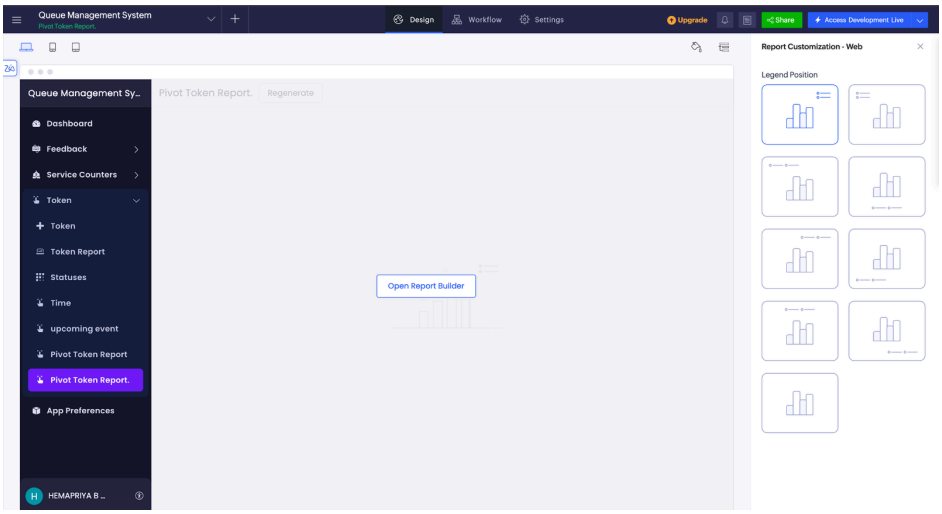


Figure 5.9: Pivot Token Report. in the Report Designer

5.4 Pages and Dashboards

The SmartQ Dashboard page (Figure 5.10) is the first entry in the menu and is built with the page builder. From top to bottom it contains a title banner, four feedback tiles, a Token Status panel, a Quick Links column whose buttons open the Add New Feedback, Add New Service Counter and Add New Token forms, two charts and the embedded All Feedback list. Table 5.4 lists the components and the values visible in the screenshot.

The values agree with the other reports. The feedback tiles show 2 escalated, 0 resolved and 3 pending, which add up to the Total Feedback of 5, and the embedded list shows five feedback records, two Escalated and three Pending (Figure 5.6). The scatter chart plots Queue Position against Estimated Waiting Time, and the bar chart shows Max Capacity by Service Type, so staff can compare the load of the service types at a glance.

Table 5.4: Components of the SmartQ Dashboard

Component	Type	Data Source	Value Observed
Count of Escalated	Count tile	Feedback	2
Count of Resolved	Count tile	Feedback	0
Count of Pending	Count tile	Feedback	3
Total Feedback	Count tile	Feedback	5
Token Status	Status panel	Token	Serving 2, Served 3, No-show 0, Waiting 9
Quick Links	Three buttons	Forms	Add New Feedback, Add New Service Counter, Add New Token
Estimated Waiting Time by Queue Position	Scatter chart	Token	Waiting time from 0 to 300; queue positions from 2 to 14
Service Type by Max Capacity	Bar chart	Service Counter	Billing about 150, Registration about 55, Other about 50 (read from the chart)
All Feedback	Embedded list	Feedback	5 records

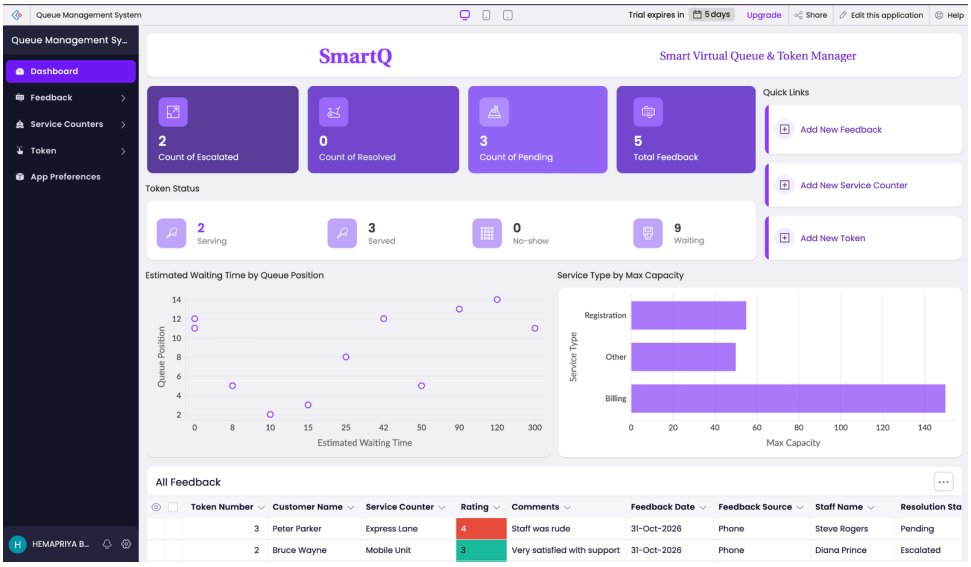


Figure 5.10: SmartQ Dashboard Page

5.5 Deluge Scripting

Deluge is the scripting language of Zoho Creator; it is used to write form-event logic, functions and integrations, for example to create or update records [2]. All nine form workflows of SmartQ run Deluge actions, and Table 5.5 lists them. Only two script bodies are visible in the supplied screenshots, Recalculate Waiting Time and Customer Assigned. For the other workflows the table gives the trigger, and the purpose is stated from the workflow name and from the outputs seen in the application.

Table 5.5: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose
DS-01	Token → Created → Load of the form (workflow "Auto-Set Token Status")	Sets the initial status of a new token

Script ID	Location / Trigger	Purpose
DS-02	Token → Created → Load of the form (workflow "Token Number Fill….")	Fills Token Number in the form SQ-nnnn (Figure 5.4)
DS-03	Token → Created → Successful form submission (workflow "Notify Customer of Wait Time")	E-mails the customer the estimated waiting time (Figure 5.16)
DS-04	Token → Created → Successful form submission (workflow "Calculate Estimated Waiting Time")	Stores the estimated waiting time of the new token
DS-05	Token → Created → Successful form submission (workflow "New_Token_Generated", 2 actions)	Posts "New token generated successfully" to Zoho Cliq (Figure 5.14); second action
DS-06	Token → Edited → Successful form submission (workflow "Update Token Status")	Runs when a token is edited
DS-07	Token → Edited → Successful form submission (workflow "Customer Assigned", 2 actions)	Posts an assignment message to Zoho Cliq
DS-08	Token → Created or Edited → Validations on form submission (workflow "Validate Queue Position")	Validates the queue position
DS-09	Service Counter → Edited → Successful form submission (workflow "Recalculate Waiting Time")	Recalculates the estimated waiting time of the waiting tokens of a counter

DS-09 – Recalculate Waiting Time. The script runs on the Service Counter form after a record is edited and saved. It acts only if the counter's average service time is not empty and is greater than zero. It fetches the Token records whose service_counter is the edited counter and whose token_status is "Waiting", sets a position counter to 1, and for each fetched token stores average_service_time multiplied by position in estimated_waiting_time before increasing the position by one. The first waiting token at the counter therefore receives one average service time, the second two, and so on, so a change to a counter's average service time refreshes the estimate of every token still waiting there (Figure 4.4). Code 5.1 reproduces the script as it appears in the editor (Figure 6.1); line breaks have been added to the two longest lines so that they fit the page. The fetch has no sort order, so the positions follow the order in which Creator returns the records; ordering the fetch by Issued At would tie the estimate to the true queue order.

Code 5.1: DS-09 Recalculate Waiting Time (Service Counter → Edited → Successful form submission)

```
if(input.average_service_time != null && input.average_service_time > 0)
{
    waitingTokens = token[service_counter == input.ID
        && token_status == "Waiting"];
    position = 1;
    for each  tokenRecord in waitingTokens
    {
        tokenRecord.estimated_waiting_time=
            input.average_service_time * position;
        position = position + 1;
    }
}
```

DS-07 – Customer Assigned. The script runs on the Token form after a record is edited and saved. It builds a text message from the token number, the customer name and the service

counter, each wrapped in ifnull so that an empty field becomes an empty string. It posts the message to the Cliq channel with the identifier smartqnotifications through the connection zoho_cliq_connection and writes the response to the log with info. Code 5.2 reproduces the first action as it appears in the editor (Figure 6.2); line breaks have been added to the first and fifth lines so that they fit the page. The workflow has a second action that is not visible. ***Code 5.2: DS-07 Customer Assigned (Token → Edited → Successful form submission)***

```
message = "A customer has been assigned to a service counter.\n"
    + "Token Number: " + ifnull(input.Token_Number,"").toString() + "\n"
    + "Customer Name: " + ifnull(input.customer_name,"").toString() + "\n"
    + "Service Counter: " + ifnull(input.service_counter,"").toString();
response = zoho.cliq.postToChannel("smartqnotifications",message,
    "zoho_cliq_connection");
info response;
```

5.6 Workflows and Automation

Nine form workflows are defined, as listed in Table 5.6 and shown in Figure 5.11 and Figure 5.12. All are enabled, and the Feedback form has none. Creator groups them by form event. Two run when a new Token form is loaded: Auto-Set Token Status and Token Number Fill…. (the trailing full stops are part of the name in the builder). They supply the initial status and the SQ-nnnn Token Number, and Issued At is also filled at this point (Figure 5.4). Three run after a new token is successfully submitted: Notify Customer of Wait Time e-mails the customer, Calculate Estimated Waiting Time stores the estimated waiting time, and New_Token_Generated (two actions) posts to Zoho Cliq. Two run after a token is edited: Update Token Status and Customer Assigned (two actions, one of which posts to Cliq). Validate Queue Position runs on both creation and edit at the validation stage, before the record is saved. On the Service Counter form, Recalculate Waiting Time runs after an edit and refreshes the waiting tokens of that counter (Section 5.5).

Table 5.6: Workflows Implemented

Workflow Name	Form	Trigger / Event	Actions	Script Ref.
Auto-Set Token Status	Token	Created → Load of the form	1	DS-01
Token Number Fill….	Token	Created → Load of the form	1	DS-02
Notify Customer of Wait Time	Token	Created → Successful form submission	1	DS-03
Calculate Estimated Waiting Time	Token	Created → Successful form submission	1	DS-04
New_Token_Generated	Token	Created → Successful form submission	2	DS-05
Update Token Status	Token	Edited → Successful form submission	1	DS-06
Customer Assigned	Token	Edited → Successful form submission	2	DS-07
Validate Queue Position	Token	Created or Edited → Validations on form submission	1	DS-08
Recalculate Waiting Time	Service Counter	Edited → Successful form submission	1	DS-09

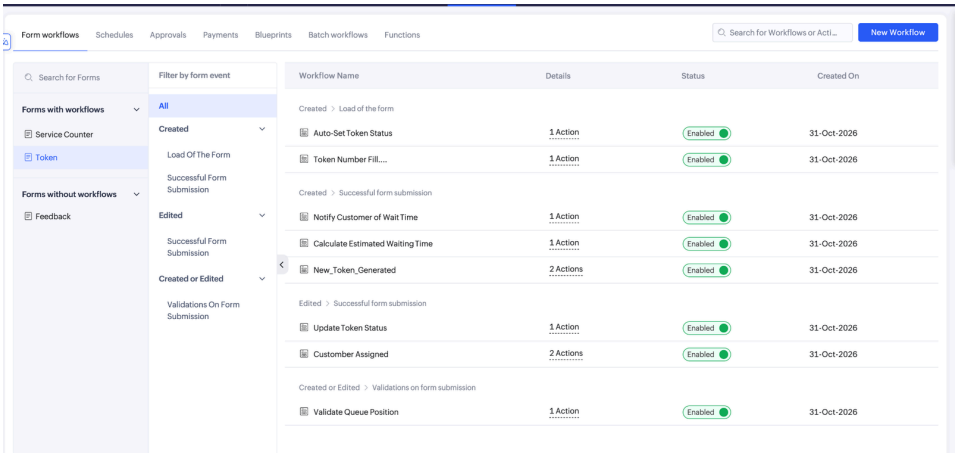


Figure 5.11: Form Workflows of the Token Form

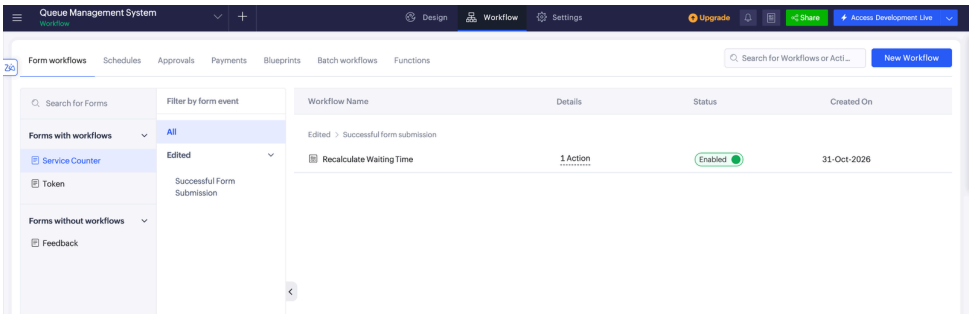


Figure 5.12: Form Workflow of the Service Counter Form

The status of a token moves from Waiting to Serving and then to Served or No-show when staff edit the record (Figure 4.3), and the Update Token Status workflow runs on that edit,No schedule, approval, blueprint, payment or batch workflow is evidenced in the supplied screenshots, so automatic marking of no-shows is not implemented. One point is recorded for follow-up: the e-mail sent after submission carries the value 95 for the estimated waiting time, equal to the value stored on the record (Figure 5.17), so the value was available when the notification ran.

5.7 Integrations

Table 5.7 summarises the integrations. **Zoho Cliq.** Two Token workflows post to Zoho Cliq with the Deluge task zoho.cliq.postToChannel through a Creator connection named zoho_cliq_connection. Figure 5.13 shows the connection: it belongs to the Admin level, its authorised account is “Zoho Cliq Account”, its status is Connected, and its References list shows the Token form, which confirms that the Token workflows use it. Messages go to the channel #smartq-notifications, a private channel with two participants, an organisation administrator and one member (Figure 5.15, personal details masked).

Figure 5.14 shows the channel after a token was created. The message begins “New token generated successfully.” and lists the token number (SQ-0014), the customer name, the service counter and the issue time, and a test message, “TEST: SmartQ is successfully connected to Zoho Cliq!”, appears below it. The Service Counter line of the message shows an internal record number instead of the counter name, because the lookup value is the record ID. The Customer Assigned script (Code 5.2) uses the same kind of expression, so its message is expected to behave in the same way.

Table 5.7: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Cliq alert: new token	Zoho Cliq channel #smartq-notifications	Deluge task zoho.cliq.postToChannel in New_Token_Generated	Connection zoho_cliq_connection	Token number, customer name, counter, issue time
Cliq alert: assignment	Zoho Cliq channel (smartqnotifications)	zoho.cliq.postToChannel in Customer Assigned (Code 5.2)	Connection zoho_cliq_connection	Token number, customer name, counter
Customer e-mail	Zoho Creator mail service	Mail action in Notify Customer of Wait Time	Creator system sender	Estimated waiting time
Form notification e-mail	Zoho Creator mail service	"New entry added" notification.	Creator system sender	Submitted token data

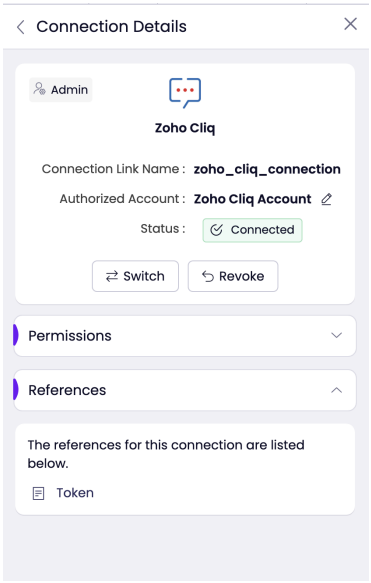


Figure 5.13: Zoho Cliq Connection Details (zoho_cliq_connection)

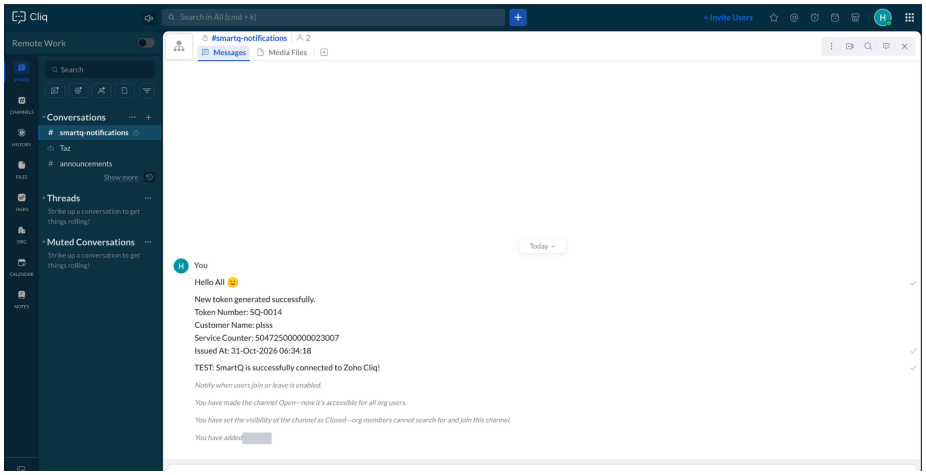


Figure 5.14: #smartq-notifications Channel in Zoho Cliq (member name masked)

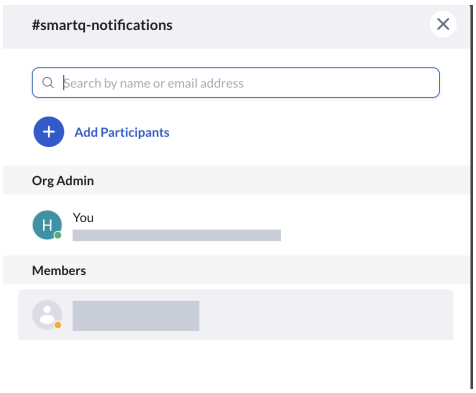


Figure 5.15: Participants of the #smartq-notifications Channel (personal details masked)

E-mail. Two e-mails were received after a token was submitted. The first has the subject "Your Token - Estimated Wait Time" and the text "Your estimated waiting time is 95 minutes."; it is sent to the e-mail address entered on the token and comes from the Notify Customer of Wait Time workflow (Figure 5.16). The second has the subject "Queue Management System : New entry added"; it is the standard Zoho Creator form notification and lists the submitted token data, among them Token Number SQ-0010, Service Counter Mobile Unit, Estimated Waiting Time 95 and Queue Position 11 (Figure 5.17; the e-mail address is masked). Both come from the Zoho Creator mail address of the trial account.

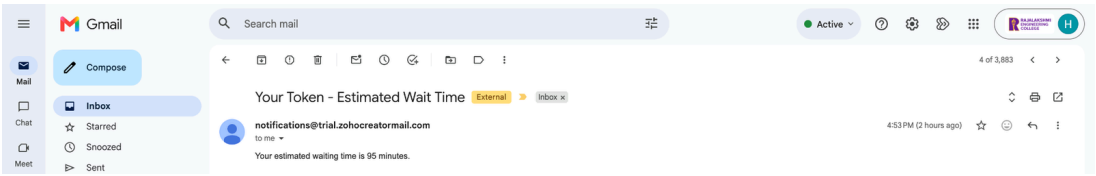


Figure 5.16: Customer E-mail "Your Token - Estimated Wait Time"

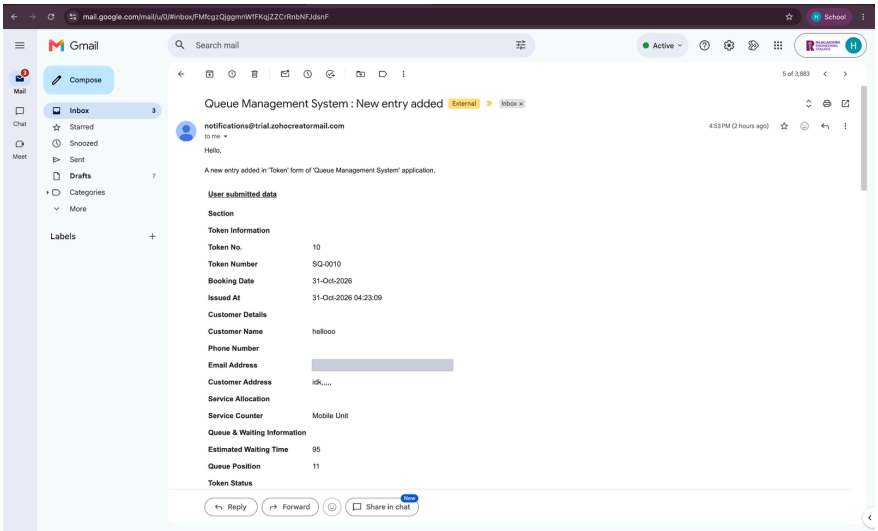


Figure 5.17: "New entry added" E-mail Notification for a Token (e-mail address masked)

5.8 User Management and Role-Based Access

Access control in SmartQ is built from roles, permission profiles and data-sharing rules, all configured under Settings → User Permissions. Figure 5.18 shows the Roles tab. The hierarchy has two levels: CEO/Administrator at the top and Service Counter Staff beneath it. In Zoho Creator a role hierarchy governs data visibility between users, whereas a permission profile governs what a user may do with forms, reports and pages. Figure 5.19 shows the three permission profiles, CEO/Administrator, Service Counter Staff and Customer. (the full stop is part of the name); their descriptions agree with the design intent recorded in Table 4.6.

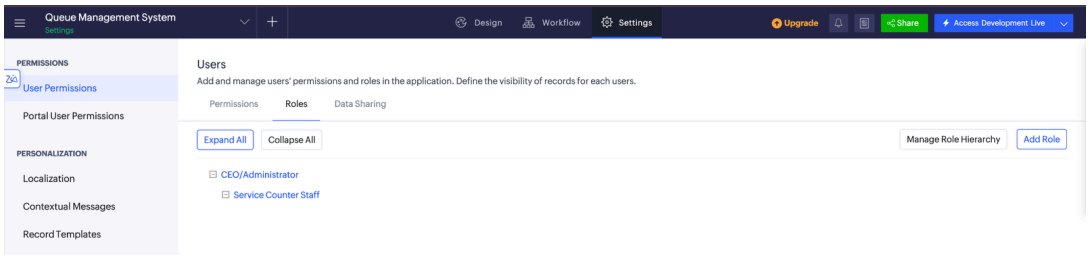


Figure 5.18: Role Hierarchy (CEO/Administrator and Service Counter Staff)

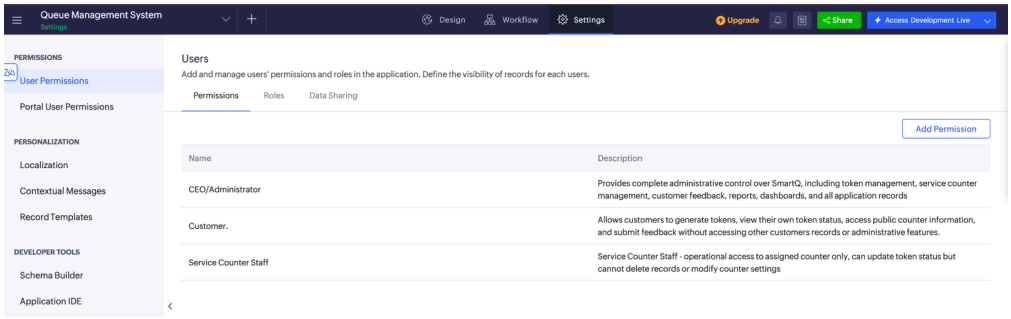


Figure 5.19: Permission Profiles of SmartQ

Figure 5.20 shows the Data Sharing tab, which holds four rules, summarised in Table 5.8. All of them are Read Only. Three share records from the Service Counter Staff role up to CEO/Administrator (Feedback, and Token twice), and one shares Service Counter records from CEO/Administrator down to Service Counter Staff, so that staff can read the counters but not change them. The rule names are cut off on screen, and the two Token rules run in the same direction and may duplicate each other. **Table 5.8:** Data Sharing Rules

Rule (as visible)	Form	Shared From	Shared To	Level of Access
Staff Feedback to Administr…	Feedback	Service Counter Staff	CEO/Administrator	Read Only
Administrator to Staff Read …	Service Counter	CEO/Administrator	Service Counter Staff	Read Only
All Token Records to Admini…	Token	Service Counter Staff	CEO/Administrator	Read Only
Staff to Administrator Read …	Token	Service Counter Staff	CEO/Administrator	Read Only

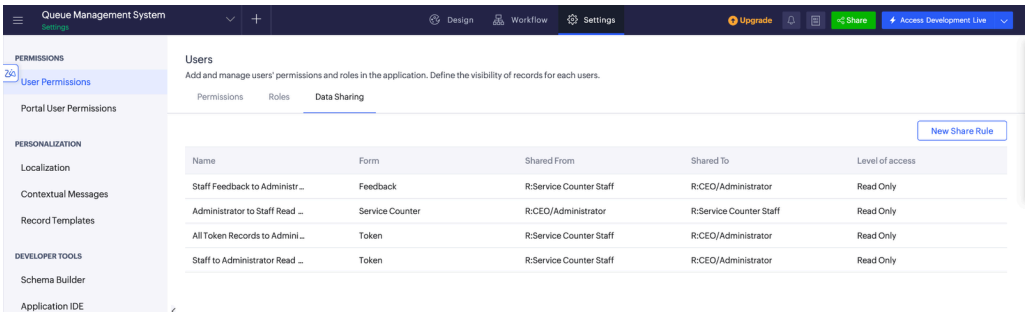


Figure 5.20: Data Sharing Rules

Figure 5.21 shows the users of the account: the super administrator and two further active users. No portal user is registered (Section 5.11). The assignment of these users to roles and profiles, a login test of each profile, and the module-level permissions (access, view, edit, delete).

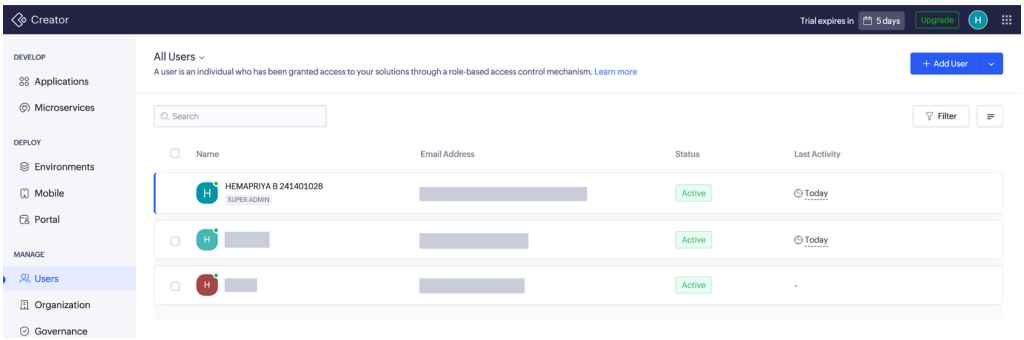


Figure 5.21: Users of the Account (e-mail addresses and names masked)

5.9 AI and Analytics Features

No generative or predictive AI feature is evidenced in SmartQ. The Zia assistant icon is visible in the builder screens, but no Zia-generated component or AI prediction is shown.

Analytics are provided by the dashboard (Figure 5.10). Four feedback tiles and a token-status panel give live counts, a scatter chart relates queue position to estimated waiting time, and a bar chart compares the maximum capacity of the service types. Two pivot report entries, Pivot Token Report and Pivot Token Report., are present in the Token section (Figure 5.9), and the Service Types Kanban report (Figure 5.7) groups the counters by service. No Zoho Analytics integration is evidenced. Further charts, such as tokens per counter and average waiting time per counter, are listed as future enhancements in Section 9.2.

5.10 Pro-Code Extension

No pro-code extension was used. The application does not use Zoho Catalyst, custom widgets or an external service written in another language. All logic is implemented with Creator workflows and Deluge, which was sufficient for the token numbering, wait estimation, validation and notification requirements. A pro-code extension would be justified only if the project later needed heavy processing, such as queue-time prediction from historical data.

5.11 Mobile and Portal Access

Customer portal. SmartQ is published as a public portal at https://smartq-qms.zohocreatorportal.in (Figure 5.22). The portal page shows the application name Queue Management System with the type Public, and it has tabs for Portal User, Authentication, Email Notification, SMS Notification and Page Customization. The customised sign-in page asks for an e-mail address and offers Next, Forgot Password and Sign Up, so customers can register themselves. A registered customer is given the Customer. permission profile (Section 5.8), which lets the customer generate a token, follow its status and submit feedback.

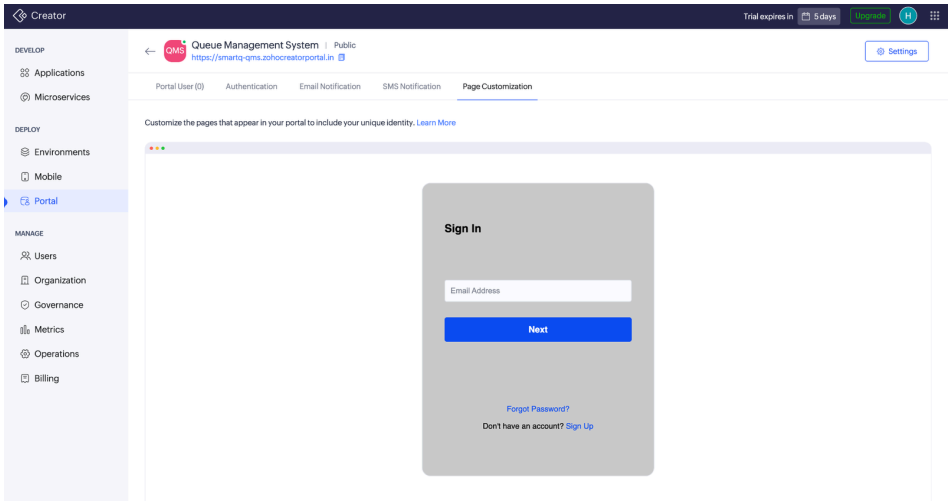


Figure 5.22: Portal Settings and Sign-In Page of the SmartQ Portal

Mobile access. The application header offers desktop, tablet and phone preview icons (for example in Figure 5.10), and the Creator console lists a Mobile entry. The platform renders applications responsively, but no screenshot of SmartQ on a phone or in the Zoho Creator mobile app was supplied.

CHAPTER 6

TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was functional and manual. SmartQ was exercised through its live interface in the Development environment of Zoho Creator (the builder header shows “Access Development Live”), and version 2.0 of the application was then published to Production on 31 October 2026 (Figure 5.1). Each feature described in Chapters 4 and 5 was checked by entering records, observing the reports and the dashboard, reading the e-mails and Zoho Cliq messages produced by the workflows, and opening the Deluge scripts in the editor. The evidence for every test is a screenshot of the running application, so the test cases describe what the screenshots show and not a separate formal test run.

Checking was done at four levels:

- **Unit level.** Single forms and fields (layout, mandatory flags, default values) and single scripts (reading the code, the Execute button and the info log).
- **Integration level.** Workflow to Zoho Cliq channel, workflow to e-mail, and the dashboard figures compared with the contents of the reports.
- **Security level.** Roles, permission profiles, data-sharing rules and the customer portal.
- **Release level.** Publication of version 2.0 to Production.

Each test case receives one of four results. **Pass (evidenced)** means that a screenshot shows the expected result. **Partly passed** means that the feature works but part of its output is wrong, and a defect has been raised. **Fail** means that the expected behaviour is absent, and a defect has been raised. **Pending Verification** means that the screenshots show the configuration but not the outcome; such a test is not assumed to pass.

The records seen in the figures are sample data (for example the customers Peter Parker, Bruce Wayne and Sarah Connor, and the counters Mobile Unit, Night Shift and Express Lane); a few records carry short test names. The tester’s own e-mail address was used for the e-mail tests and is masked in the figures.

6.2 Test Cases

Table 6.1 lists the 30 test cases, grouped by feature. “Pass (evidenced)” means that the cited figure demonstrates the expected result; it does not imply that a separate formal test run was recorded.

Table 6.1: Test Cases

TC ID	Feature	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC01	Service counters	Create service counters	Fill the Service Counter form (name, type, status, capacity, hours, manager, contact, average time) and submit	Records are saved and listed	Five counters appear on the Service Types board: Mobile Unit and Night Shift (Registration); harini, Express Lane and Specialist Support (Billing) (Figure 5.7)	Pass (evidenced)
TC02	Service counters	Mandatory validation	Submit the Service Counter form with Counter Name, Service Type, Counter Status or Average Service Time empty	A validation message is shown	The four fields carry the mandatory asterisk (Figure 5.2); a blocked submission was not captured	Partly passed
TC03	Service counters	Operating-hours subform	Add rows with location, day, opening and closing time and closed flag	Rows are saved with the counter	The subform with Add New and delete-row controls is shown empty (Figure 5.2); a saved row was not captured	passed
TC04	Token issue	Defaults on form load	Open the Token form	Token Number and Issued At are filled before the user types	Token Number SQ-0015 and Issued At 31-Oct-2026 07:13:03 are already present (Figure 5.4)	Pass (evidenced)
TC05	Token issue	Counter required	Check the validation of the Service Counter field on the Token form	A token cannot be saved without a counter	The field is not mandatory (Figure 5.3), so a token can be saved without a counter (D-09)	Partly passed
TC06	Token issue	Sequential token numbers	Create tokens one after another	Numbers increase without repetition	SQ-0010 (04:23), SQ-0014 (06:34) and SQ-0015 (07:13) rise with time (Figures 5.4, 5.14 and 5.17)	Pass (evidenced)
TC07	Token issue	Issue time recorded	Submit a token and read Issued At	Issued At holds the date and time of issue	Issued At is present on the form, in the e-mail and in the Cliq message, but its clock is 12 h 30 min behind local time (D-11)	Partly passed (D-11)
TC08	Token status	Status values in use	Read the Token Status tiles on the dashboard	Waiting, Serving, Served and No-show are all available	Tiles show Serving 2, Served 3, No-show 0 and Waiting 9 (Figure 5.10); a stored token holds Waiting (Figure 5.8)	Pass (evidenced)
TC09	Token status	Status change by staff	Edit a token and change its status	The new status is saved and the Update Token Status workflow runs	The workflow is enabled (Figure 5.11); the edit and its effect were not captured	Passed
TC10	Waiting time	Queue position and estimate stored	Open a stored token	Estimated Waiting Time and Queue Position hold values	Night Shift token: queue position 3 and estimated waiting time 15 (Figure 5.8)	Pass (evidenced)
TC11	Waiting time	Estimate follows the rule	Compare the estimate with average service time × queue position	The estimate equals average service time × position for every token	15 at position 3 implies 5 minutes per customer; 95 at position 11 (Figures 5.16 and 5.17) does not divide evenly (D-10); the counters' average times are not shown	Partly passed
TC12	Waiting time	Recalculate and validate workflows	Edit a counter's average service time; submit a token with a queue position	Waiting tokens are recalculated and the position is validated	Recalculate Waiting Time (Figure 5.12) and Validate Queue Position (Figure 5.11) are enabled and the first script is visible (Figure 6.1); no run was captured	Partly passed

TC ID	Feature	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC13	Feedback	All Feedback report	Open All Feedback	Feedback records are listed with all fields	Five records with token number, customer, counter, rating, comments, date, source, staff and status (Figure 5.6)	Pass (evidenced)
TC14	Feedback	Feedback form	Open the Feedback form	Four sections are shown; Token Number and Feedback Date are mandatory	Token & Service Details, Customer Feedback, Staff & Service Review and Follow-Up Details are present; both fields carry the asterisk (Figure 5.5)	Pass (evidenced)
TC15	Dashboard	Feedback tiles agree with report	Compare the four feedback tiles with the All Feedback rows	Counts match	Escalated 2, Resolved 0, Pending 3 and Total 5 (Figure 5.10) equal the status counts of the five rows (Figure 5.6)	Pass (evidenced)
TC16	Dashboard	Token tiles agree with token reports	Compare the token tiles with the Token Report and Statuses	Counts match	The tiles total 14 tokens; the Token Report and Statuses outputs were not supplied	Partly passed
TC17	Dashboard	Charts render	Open the dashboard	Both charts show data	The scatter chart of waiting time by queue position (11 points) and the bar chart of maximum capacity by service type (Registration, Other, Billing) are drawn (Figure 5.10)	Pass (evidenced)
TC18	Dashboard	Quick links	Click Add New Feedback, Add New Service Counter and Add New Token	Each opens its form	Three links are present (Figure 5.10); clicking them was not captured	Partly passed
TC21	Reports	Service Types board	Open Service Types	Counters are grouped by service type	Registration holds 2 cards and Billing 3; Document Verification and Consultation show “No records” (Figure 5.7)	Pass (evidenced)
TC22	Integration	Cliq connection	Open the connection details	The connection is authorised and used	zoho_cliq_connection is Connected under the Zoho Cliq account and its reference is the Token form (Figure 5.13)	Pass (evidenced)
TC23	Integration	New-token message in Cliq	Create a token	A message with the token details appears in #smartq-notifications	The message for token SQ-0014 shows the customer and issue time, but the counter appears as the number 504725000000023007 (Figure 5.14; D-07)	Partly passed (D-07)
TC24	Integration	Customer-assigned message	Assign a counter to a token by editing it	One message appears for the assignment	The script is visible (Figure 6.2); no such message is shown in the channel, and the workflow has no visible condition (D-08)	Partly passed
TC25	Notification	New-entry e-mail	Submit a token	A notification with the submitted data arrives	“Queue Management System : New entry added” lists token SQ-0010 and its fields (Figure 5.17)	Pass (evidenced)
TC26	Notification	Waiting-time e-mail	Submit a token with an e-mail address	The customer receives the estimate	“Your Token - Estimated Wait Time” states 95 minutes, equal to the stored estimate (Figure 5.16)	Pass (evidenced)
TC27	Security	Roles, profiles and sharing	Open User Permissions	Roles, profiles and sharing rules are configured	A two-level role hierarchy, three profiles and four read-only sharing rules are listed (Figures 5.18 to 5.20); two rules overlap (D-12)	Pass (evidenced)

TC ID	Feature	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC28	Security	Staff restrictions enforced	Sign in as Service Counter Staff and try to delete a record or edit a counter	The action is refused	The profile description states the limits (Figure 5.19); no sign-in test was captured	Partly passed
TC29	Security	Customer portal	Sign in at the portal address as a customer, issue a token and view own records	The customer sees only own records	The sign-in page loads (Figure 5.22) but the portal has 0 users, so no customer test was possible	Partly passed
TC30	Release	Publish to Production	Publish version 2.0	Publication is confirmed	Zoho Creator confirms Queue Management System 2.0 published on 31 Oct 2026 at 19:24 IST (Figure 5.1)	Pass (evidenced)

Table 6.2 summarises the results. No test case failed because of a script error; the three failures are design gaps (D-09, D-13 and D-16 in Section 6.4 give the detail). The pending tests are pending because the screenshots show a configuration but not its outcome, and they close as soon as the missing screenshot or sign-in test is captured. The screenshots cited in Table 6.1 are shown once, in Chapter 5, and are not repeated here.

6.3 Debugging Deluge Scripts

Three debugging aids were used: the Execute button of the Deluge editor, the info statement that writes a value to the log, and the output that each workflow leaves behind (a Cliq message or an e-mail). Two script bodies are now visible in the editor, and the workflow list makes it possible to trace which script produced which output.

Recalculate Waiting Time.

This workflow runs when a Service Counter record is edited and saved (Figure 5.12). The script is listed in Code 5.1 and shown in the editor in Figure 6.1. If the counter's average service time is set and greater than zero, the script fetches the tokens of this counter whose status is Waiting, starts a position counter at 1 and, for each token, stores the average service time multiplied by the position in Estimated Waiting Time before increasing the position. Reading the code gives three points to test. First, the fetch has no sort order, so positions follow the platform's default record order and not necessarily the order of Issued At. Second, the script writes only the estimated waiting time and does not change Queue Position. Third, the first waiting token is given one full service time even when nobody is being served.



Figure 6.1: Deluge Script of the Recalculate Waiting Time Workflow

Customer Assigned.

This workflow runs on every successful edit of a Token record (Figure 5.11). The script is listed in Code 5.2 and shown in the editor in Figure 6.2. It builds a four-line message from the token number, customer name and service counter, posts it to the channel with the unique name smartqnotifications through the connection zoho_cliq_connection, and writes the response to the log. Two points came out of reading the script. The service counter is a lookup field, and converting a lookup value to text gives the record ID; this is the likely cause of the number shown in Figure 5.14 (D-07), although the script of the new-token workflow is not visible. The script also has no condition, so the message “A customer has been assigned to a service counter” may be posted on any edit, for example when only the status changes (D-08). Code 6.1 shows a proposed correction of the counter name; it has not been deployed or tested.



Figure 6.2: Deluge Script of the Customer Assigned Workflow

Table 6.3 lists the symptoms that were traced in this way and what each trace showed.

Table 6.3: Debugging Observations

No.	Symptom	Technique Used	Finding	Defect
1	The counter appears as 504725000000023007 in the Cliq message	Read the posted message and the sibling script	A lookup value converted to text gives the record ID, not the counter name	D-07
2	Estimated waiting time 95 at queue position 11	Compared the stored values with the formula of the recalculation script	The formula gives a whole multiple of the average time and 95 ÷ 11 is not whole, so another rule or hand entry is involved	D-10
3	Issued At 04:23:09 on an e-mail delivered at 4:53 PM	Compared the time stamp with the delivery time	The gap is 12 h 30 min, the difference between UTC−7 and IST.	D-11
4	No customer-assigned message in the channel	Read the workflow trigger and script	The trigger is every edit and the script has no condition	D-08

No.	Symptom	Technique Used	Finding	Defect
5	The same token number on feedback for different customers	Opened the token record and compared it with the feedback rows	Feedback has no workflow, so customer and counter are not taken from the token	D-16
6	Empty Token Number on one stored token	Opened the record	Token No. 1 holds a value but Token Number is empty; the record predates the Token Number Fill workflow.	D-15

6.4 Defect Log

Table 6.4 records all issues. The six defects D-01 to D-06 come from the first version of this report and have been re-checked against the current screenshots; all six are now closed, with one re-test pending. Ten new defects, D-07 to D-16, were found while preparing the evidence for this chapter and are open. The most important are D-09 (a token can be saved without a counter), D-16 (feedback is not tied to its token) and D-07 (the counter is not readable in the Cliq message). Figure 5.3 shows the field settings behind D-09.

Table 6.4: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
D-01	Estimated Wait was empty for eight of nine waiting tokens (first build).	Suspected: the counter was looked up with the wrong field in the first script	Estimation rebuilt as the workflows Calculate Estimated Waiting Time and Recalculate Waiting Time; values are now stored (Figures 5.8 and 5.16).	Closed; re-test all records
D-02	Issued At was empty for one token.	Not established.	Issued At is filled when the form loads (Figure 5.4).	Closed
D-03	One token was dated 22-Oct-2026 among 20-Oct-2026 records.	Probably a manual edit in the old test data.	Old test records no longer appear; not reproduced in the current screenshots.	Closed; not reproduced
D-04	The Staff Access description did not match its permissions.	Incomplete profile configuration.	Replaced by three profiles with matching descriptions (Figure 5.19).	Closed; sign-in re-test pending (TC28)
D-05	The third dashboard tile and the figure “11” had no label.	Label not set.	All tiles are now labelled (Figure 5.10).	Closed
D-06	The Kanban report was called “Status of Project” but shows token status.	Name not aligned with content.	The menu now shows “Statuses” under Token.	Closed
D-07	The Cliq message shows the Service Counter as the number 504725000000023007 (Figure 5.14).	The lookup value is converted to text, which gives the record ID and not the counter name.	Proposed: read Counter Name from the Service Counter record (Code 6.1).	Open
D-08	The Customer Assigned workflow runs on every edit and its script has no condition, so its message may be posted when only a status changes (Figure 6.2).	Trigger is Edited → Successful form submission with no visible condition.	Proposed: post the message only when a counter is newly assigned.	Open
D-09	The Service Counter lookup on the Token form is not mandatory and allows new entries (Figure 5.3).	Field properties not tightened: Mandatory is off and Allow new entries is on.	Proposed: make the field mandatory and switch off Allow new entries.	Open
D-10	An estimated waiting time of 95 at queue position 11 does not follow average time × position (Figures 5.16 and 5.17).	To be established: another rule or hand entry; the recalculation script does not update Queue Position (Code 5.1).	None recorded.	Open

Bug ID	Description	Root Cause	Fix Applied	Status
D-11	Issued At shows a time 12 h 30 min behind local time (04:23:09 on an e-mail delivered at 4:53 PM).	Application time zone probably UTC−7 and not IST.	Proposed: set the application time zone to Asia/Kolkata.	Open
D-12	Two read-only rules share Token records from Service Counter Staff to CEO/Administrator (Figure 5.20).	Duplicate rule created during configuration.	Proposed: delete one of the two rules.	Open
D-13	Stray full stops in names: the report “Pivot Token Report.” (empty, same name as the first pivot report) and the profile “Customer.” (Figures 5.9 and 5.19).	Names typed with a trailing full stop; the second pivot report was never configured.	Proposed: configure or delete the empty report and rename the profile.	Open
D-14	Ratings and comments do not agree on one scale: a rating of 4 accompanies “Staff was rude” and a rating of 2 accompanies “Great experience overall” (Figure 5.6).	The rating scale has no stated direction; the sample data may be inconsistent.	Proposed: label the choices (for example 1 = Very poor to 5 = Excellent) and correct the sample data.	Open
D-15	One token (Sarah Connor, Token No. 1) has an empty Token Number, and its Issued At (28-Oct) is earlier than its Added Time (31-Oct) (Figure 5.8).	Record probably added by hand or imported before the Token Number Fill workflow existed.	Proposed: fill the number or remove the record before go-live.	Open
D-16	Feedback is not tied to its token: token 2 appears for two customers and counters, token 4 likewise, and Sarah Connor’s feedback names token 4 at Main Service Desk while her token is No. 1 at Night Shift (Figures 5.6 and 5.8).	Customer Name and Service Counter are not derived from the selected token, and Feedback has no workflow (Figure 5.11).	Proposed: fill both fields from the token lookup and make them read-only.	Open

6.5 User Acceptance Testing

Feedback from independent users has not yet been collected. The user list (Figure 5.21) shows three accounts: the super administrator, a second account named Hemapriya and a user named Harini B. Harini B is a member of the #smartq-notifications channel (Figure 5.15) but has no recorded activity, and the portal has no customer users (Figure 5.22), so no acceptance test by another person is evidenced. Table 6.5 is to be completed after at least two users have followed the scenario below. The application header shows that the trial edition has five days left (Figure 5.10), so the scenario should be run before the trial ends.

Acceptance scenario:

1. A customer, or a staff member on the customer’s behalf, submits the Token form with a name, an e-mail address and a service counter.
2. The customer checks the e-mail “Your Token - Estimated Wait Time” and the new-entry notification.
3. Staff check the message in #smartq-notifications and find the token on the Statuses board.
4. Staff change the status to Serving and then to Served, and the dashboard tiles are read again.
5. The customer submits feedback for the token, and the administrator checks the feedback tiles and the All Feedback report.

CHAPTER 7

DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

Zoho Creator separates an application into Development, Stage and Production environments: changes are made and tried in Development and then published forward, so that real users work on a stable copy. The Environments entry is part of the console navigation (Figure 5.21). SmartQ is built in an application named “Queue Management System”, which carries the SmartQ brand on its dashboard page. After development, version 2.0 of the application was published to Production on 31 October 2026 at 19:24 IST (Figure 5.1).

The deployment followed the steps below. Steps 1 to 4 are supported by the evidence; steps 5 and 6 are listed for completeness.

1. Develop and unit-test forms, reports, the dashboard and workflows in the Development environment (the builder with “Access Development Live”).
2. Create and authorise the Zoho Cliq connection (zoho_cliq_connection) and test the notification scripts (Chapter 5, Section 5.7).
3. Configure roles, permission profiles and data-sharing rules (Section 7.3).
4. Publish version 2.0 to Production (Figure 5.1).
5. Share the portal URL with customers and staff and create their accounts
6. Record release notes and review the first week of use..

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Status
1	All test cases executed in Development / Stage	Test cases are recorded in Chapter 6.
2	Sample / test data removed or isolated	Not done: sample tokens (for example customer names “hellooo” and “plsss”) and sample feedback records are visible in the screenshots.
3	Roles, permissions and users configured	Done: 2 roles, 3 permission profiles, 4 data-sharing rules and 3 users exist (Figures 5.18 to 5.21); assignment of users to profiles.
4	Connections authorised	Done: Zoho Cliq connection shows status Connected (Figure 5.13)
5	Schedules and workflows enabled	Nine form workflows are enabled (Chapter 5, Section 5.6); no schedule workflow is evidenced
6	Application published and URL verified	Published as version 2.0 on 31 Oct 2026 (Figure 5.1); URL https://smartq-qms.zohocreatorportal.in.
7	Version / release notes recorded	Version number 2.0 appears in the publish e-mail.

7.2 Performance Optimisation

No performance measurement was recorded for SmartQ, so no before-and-after figures are reported. The review of the scripts identified the opportunities in Table 7.2. They are proposals. The script that recalculates waiting times already filters at the platform: it fetches only the tokens whose service counter equals the edited counter and whose status is Waiting, instead of fetching every token and testing the counter inside a loop. The remaining opportunities concern when the script runs and in what order the records are processed.

Table 7.2: Performance Optimisation Opportunities (Proposed, Not Measured)

Proposed Optimisation	Component	Current Approach	Proposed Approach	Expected Benefit
Run the recalculation only when the average service time changes	Recalculate Waiting Time (Service Counter)	Runs on every successful edit of a Service Counter record; the script tests only that the average service time is greater than zero.	Add a condition so that the script runs only when Average Service Time is changed	Fewer unnecessary updates of waiting tokens
Make the processing order explicit	Recalculate Waiting Time	Queue position is assigned in the order in which the fetch returns the records; no sort is specified	Sort the fetch by the time the token was added (Code 7.1)	Queue position follows order of arrival
Default filter on reports	Token Report and other token reports	Reports show all tokens	Apply a default criterion such as the current booking date	Shorter load time as history grows
Archive served tokens	Token form	All tokens stay in the active data set	Move old Served and No-show tokens to an archive report	Smaller working data set

7.3 Security Features

Security in SmartQ is built from the platform controls described below. Table 7.3 summarises them and the remainder of the section gives the evidence.

Authentication. Customers reach the portal through the sign-in page in Figure 5.22 (e-mail address, then Next). A Sign Up link is offered on the same page, so self-registration is possible; the permission profile given to a new portal user therefore matters and is described below.

Roles and permission profiles. The role hierarchy (Figure 5.18) has two levels: CEO/Administrator, with Service Counter Staff beneath it. Three permission profiles are defined (Figure 5.19 and Table 4.6). A role hierarchy controls how far a user can see the records of the roles beneath, whereas a permission profile controls what the user may do with each form, report and page. The form-by-form permission settings of each profile are listed in the permission matrix in Chapter 4, Section 4.6.

Data sharing. The four read-only rules in Table 5.8 (Figure 5.20) let the administrator read the Feedback and Token records of staff and let staff read the Service Counter records. The two Token rules have the same form, source and target, so one of them may be redundant.

Scoped connection and closed channel. The notification scripts post to Zoho Cliq through the connection zoho_cliq_connection. Figure 5.13 shows it with the status Connected and with the Token form listed under References, which means that the Token workflows use it. The channel #smartq-notifications is a closed channel: the channel history records that its visibility was set to Closed so that other members of the organisation cannot search for and join it. Only two participants are listed (Figure 5.15): the organisation administrator and one member.

Personal data. The Token form collects the customer's name, e-mail address, phone number and address, and the Feedback form records comments and staff names. The Field Properties panel of Zoho Creator offers Data Privacy (contains personal data) and Data Security (encrypt data) options; they were not selected for the fields shown in the screenshots. Marking the customer fields as personal data and considering encryption is recommended. The "New entry added" e-mail (Figure 5.17) repeats the submitted customer details, so its recipients should be limited to administrators. Credentials, keys and personal contact details have been excluded or masked in this report.

Table 7.3: Security Controls

Security Control	Where Applied	Threat Mitigated	Status
Portal sign-in (e-mail address and password)	Portal → Page Customization / Authentication	Unauthenticated access	Implemented (Figure 5.22)
Role hierarchy (CEO/Administrator > Service Counter Staff)	Settings → User Permissions → Roles	Visibility of records across organisational levels	Implemented (Figure 5.18)
Permission profiles (3)	Settings → User Permissions → Permissions	Staff changing counter settings; customers reading others' records	Implemented (Figure 5.19); form-level settings in Section 4.6
Data-sharing rules (4, read only)	Settings → User Permissions → Data Sharing	Uncontrolled access to Feedback, Token and Service Counter records	Implemented (Figure 5.20); possible duplicate rule
Authorised connection	zoho_cliq_connection	Use of unauthorised accounts for messaging	Implemented (Figure 5.13)
Closed Cliq channel	#smartq-notifications	Notification content visible to the whole organisation	Implemented (Figure 5.15)
Personal-data marking and field encryption	Customer Name, Phone, Email, Address	Exposure of personal data	Proposed
Record-level criteria for customers (own tokens only)	Customer permission profile	A customer reading another customer's token	Stated in the profile description.

7.4 Governance and Administration

Governance in Zoho Creator is exercised from the console, whose navigation (Figure 5.21) offers Users, Organization, Governance, Metrics, Operations and Billing. The Users page lists three users, all Active: the account owner, who holds the Super Admin designation, and two further users.The supplied evidence does not show the use of the audit trail, governance reports or metrics, so none is claimed.

The following practices are recommended for the production deployment:

- **Review.** Review the role hierarchy, the permission profiles and the data-sharing rules at a fixed interval, and remove any redundant rule.
- **Connections.** Keep the Zoho Cliq connection owned by an administrator account and review its references after every change to a workflow.
- **Test data.** Remove sample tokens and feedback before go-live and keep a separate test counter for later trials.
- **Release notes.** Record each published version (the first is 2.0) with the workflows and scripts that changed.

CHAPTER 8

RESULTS AND DISCUSSION

8.1 Application Output

A single scenario illustrates the working application. An administrator configures service counters on the Service Counter form, which holds the counter name, service type, counter status, maximum capacity, a Location and Operating Hours table, the manager's name and contact number and the average service time (Chapter 5, Section 5.2). A customer or a member of staff then opens the Token form. The Token Number (for example SQ-0015) and the Issued At date and time are already filled when the form opens; the user enters the customer name, e-mail address, phone number and address, and chooses a service counter. On submission the application stores the token with the status Waiting, a Queue Position and an Estimated Waiting Time. Figure 5.8 shows a stored token: it is at Night Shift with queue position 3 and an estimated waiting time of 15, which equals three times an average service time of 5.

Three notifications follow a new token. First, an e-mail titled "Your Token – Estimated Wait Time" tells the customer the waiting time (Figure 5.16; here, 95 minutes).Second, the platform sends a "New entry added" e-mail that lists the submitted data, including the queue position and waiting time (Figure 5.17). Third, a message "New token generated successfully" with the token number, customer name, service counter and issue time is posted to the Zoho Cliq channel #smartq-notifications (Figure 5.14) The Cliq message shows the service counter as a long number instead of its name; this is recorded as defect D-07 in Table 6.4.

When a token is edited and assigned to a counter, the workflow Customer Assigned posts the message "A customer has been assigned to a service counter" with the token number, customer name and counter to the same channel (Appendix C, Section C.2). When the average service time of a counter is edited, the workflow Recalculate Waiting Time sets the estimated waiting time of every waiting token of that counter to the average service time multiplied by the token's position in the waiting list (Appendix C, Section C.1).

Staff and administrators see the queue on the SmartQ Dashboard (Figure 5.10); its tiles, charts and quick links are described in Section 5.4.

The Service Types board (Figure 5.7) groups the counters by service type, so an administrator can see at once how many counters offer each service. Four columns are visible in the screenshot (Registration, Document Verification, Billing, Consultation,etc).

Feedback is collected on the Feedback form and listed in All Feedback (Figure 5.6); the five sample records are the source of the tile counts on the dashboard.

8.2 Achievement of Objectives

Table 8.1 maps the objectives of the project to the outcomes. [ALIGN WITH SECTION 1.3]

Table 8.1: Mapping of Objectives to Outcomes

Objective	Outcome	Status
Design linked Service Counter, Token and Feedback forms	Three forms were built; the Token form refers to a Service Counter and the Feedback form to a token (Chapter 4 and Chapter 5).	Achieved
Automate token numbering, issue time, queue position and waiting time	Token Number and Issued At are pre-filled, and queue position and estimated waiting time are stored (Figure 5.8). Results of the test cases are in Chapter 6.	Achieved
Provide reports and a dashboard	Kanban board, list reports, feedback list and a dashboard with tiles and two charts are in place (Figures 5.6, 5.7 and 5.10). The Pivot Token Report has no output yet (D-13).	Achieved; one report incomplete
Notify customers and staff	Two e-mails and Cliq messages were received (Figures 5.14, 5.16 and 5.17). The Cliq message shows a counter number and not its name (D-07). SMS is not evidenced.	Partly achieved
Collect and track customer feedback	Feedback form, All Feedback list and dashboard counts exist (Figures 5.6 and 5.10). The rating scale needs a check (D-14).	Achieved; verification required
Control access with roles, profiles and a customer portal	Two roles, three profiles and four data-sharing rules exist and the portal is published (Figures 5.18 to 5.20 and 5.22). Portal users and login tests are not evidenced.	Partly achieved; verification required
Deploy the application	Version 2.0 was published to Production on 31 Oct 2026 (Figure 5.1).	Achieved

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Aspect	Existing (Manual) System	SmartQ Application
Token issue	Paper slip or register entry	Form submission with an automatic token number (SQ-0015 format)
Time of issue	Not recorded or written by hand	Issued At filled by the application
Waiting-time information	None	Queue position and estimated waiting time stored, and e-mailed to the customer
Queue overview	Visual count of the line	Dashboard tiles, charts, Kanban board and token reports

Aspect	Existing (Manual) System	SmartQ Application
Status tracking	Mental note or tick in a register	Token statuses Waiting, Serving, Served and No-show with counts on the dashboard
Staff alert	None	Cliq messages for new tokens and for customer assignment
Customer access	None	Portal with sign-in, token creation and own-token view (profile Customer).
Feedback	Informal	Feedback form with rating, source, staff name, resolution status and follow-up flag
Access control	Physical presence	Roles, permission profiles and data-sharing rules

8.4 Limitations of the Low-Code Approach

Several limitations were met in this project. First, platform behaviour that is easy to configure can be hard to inspect. The Cliq message that prints a counter number instead of a name (D-07) shows that event-driven logic depends on knowing exactly what a lookup field returns inside a script, which matches the finding that event handling is a difficult area of low-code development [5]. Second, the wait estimate is a simple model: the queue position multiplied by the counter's average service time. It is recalculated when a counter is edited, but it does not account for the time already spent on the token being served, for priorities or for the real arrival rate Third, the notifications come from the platform's trial sender, the SMS channel is not evidenced and the trial ends within days, so the features available in a live service depend on the subscription. Fourth, customisation is bounded by the platform's components, a trade-off also reported by practitioners [4], and the application depends on the vendor's hosting, pricing and feature availability. Where such limits become binding, pro-code extensions are the workaround (Chapter 5, Section 5.10).

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Most of SmartQ sits at the no-code end of the spectrum: the three forms, the lookups, the Location and Operating Hours table, the Kanban and list reports, the dashboard tiles and charts, the portal, the roles, the permission profiles and the data-sharing rules were produced by configuration alone. The nine form workflows are low-code, because they use short Deluge scripts to compute queue positions and waiting times, to build messages and to post them to Zoho Cliq. No pro-code component was used. The project therefore shows that a useful queue manager with customer access, notifications and feedback can be built with a very small amount of code, and that the points where code is needed are exactly those that carry business rules (calculation, validation and notification).

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

SmartQ – Smart Virtual Queue & Token Manager demonstrates a working low-code solution for digital queue management on Zoho Creator. The deployed application (Queue Management System, version 2.0) provides three forms (Service Counter, Token and Feedback), reports for tokens, counters and feedback, a dashboard with status tiles and charts, nine form workflows, a Zoho Cliq connection, e-mail notifications, two roles, three permission profiles, four data-sharing rules and a customer portal. Tokens receive a number, an issue time, a queue position and an estimated waiting time, and move through the statuses Waiting, Serving, Served and No-show. Customers and staff are informed by e-mail and by messages in a closed Cliq channel.

The review of the final screenshots also identified work that remains (Table 6.4): the service counter is printed as a number in the Cliq message, the rating scale and the sample feedback disagree, one data-sharing rule may be redundant, the Pivot Token Report has no output, and the trial period is about to end. These items should be closed and re-tested before the application is relied upon in a live service area.

9.2 Future Enhancements

- **Readable notifications.** Use the counter name in every Cliq message and add the queue position and estimated waiting time to it (D-07).
- **SMS notification.** Send an SMS to the customer when the token is created and when it is near the front of the queue, using the phone number that is stored on the token and the SMS Notification option of the portal.
- **Improved waiting-time model.** Recalculate whenever a token changes status, add the time left for the token being served, and support priority tokens and several counters per service type.
- **Scheduled workflow.** Mark tokens as No-show automatically after a defined time and send reminders for tokens booked for a later date.
- **Analytics.** Complete the Pivot Token Report and add charts for tokens per counter, average waiting time, no-show rate and average feedback rating per counter.
- **Customer self-service.** Add a portal page that shows the customer’s position in the queue and allows a booking date to be chosen.

- **Mobile access.** Configure the Zoho Creator mobile app for staff and customers.
- **Privacy hardening.** Mark customer name, phone, e-mail and address as personal data, consider field encryption and limit the recipients of the "New entry added" e-mail.
- **Multi-branch operation.** Add a branch field so that several locations can share one application.

9.3 Learning Outcomes and Career Relevance

The project exercises the main skills of the course: modelling data with forms, lookups and a subform, building reports and a dashboard, scripting with Deluge, configuring form workflows, integrating Zoho Cliq, configuring roles, permission profiles and data sharing, publishing a portal and deploying a version to Production. It also shows the importance of checking automation against real records, as the defects in Table 6.4 demonstrate. These skills are relevant to roles in business-application development, ERP and workflow automation, and low-code or citizen-development support.

REFERENCES

[1] Zoho Corporation, "Zoho Creator Resource Center." [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 20 Oct. 2026).

[2] Zoho Corporation, "Introduction to Deluge," Zoho Deluge Help. [Online]. Available: https://www.zoho.com/deluge/help/ (Accessed: 20 Oct. 2026).

[3] Zoho Corporation, "Cliq Help Center." [Online]. Available: https://www.zoho.com/cliq/help/ (Accessed: 20 Oct. 2026).

[4] Y. Luo, P. Liang, C. Wang, M. Shahin and J. Zhan, "Characteristics and Challenges of Low-Code Development: The Practitioners' Perspective," in Proc. 15th ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement (ESEM), 2021. [Online]. Available: https://arxiv.org/abs/2107.07482 (Accessed: 20 Oct. 2026).

[5] M. A. Al Alamin, S. Malakar, G. Uddin, S. Afroz, T. B. Haider and A. Iqbal, "An Empirical Study of Developer Discussions on Low-Code Software Development Challenges," arXiv:2103.11429, 2021. [Online]. Available: https://arxiv.org/abs/2103.11429 (Accessed: 20 Oct. 2026).

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

The checklist reflects the supplied evidence. “Partial” marks features that are only partly evidenced. Examiners should verify the entries against the live application.

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created and sections / menu organised	Y	Menu: Dashboard, Feedback, Service Counters, Token, App Preferences	5.1	Figures 4.6 and 5.10
II	Forms with ≥ 8 distinct field types	Y	Single Line, Drop Down, Radio, Number, Auto Number, Date, Date-Time, Time, Email, Phone, Address, Multi Line, Check box, Lookup (types read from the forms)	5.2	Figures 5.2 to 5.5
II	Lookup field (relationship between forms)	Y	Token → Service Counter; Feedback → Token.	4.2, 5.2	Figures 4.2 and 5.3
II	Subform (parent–child data)	Y	Location & Operating Hours table on Service Counter	5.2	Figure 5.2
II	Field validations / layout rules	Y	Mandatory fields and the workflow Validate Queue Position; layout rules not evidenced	5.2, 5.6	Tables 4.2 to 4.4; Section C.7
II	List report with filter, grouping, custom action	Y	All Feedback (colour-coded rating), All Service Counters, Token Report; grouping and custom actions not evidenced	5.3	Figure 5.6
II	Kanban / Calendar / Timeline / Map report	Y	Service Types (Kanban); Feedback Board and upcoming event.	5.3	Figure 5.7
II	Pivot table / chart report	Y	Pivot Token Report created; output not yet generated (D-13)	5.3	Figure 5.9
II	Page / dashboard built with UI Builder	Y	Dashboard with tiles, two charts, quick links and embedded list	5.4	Figure 5.10
II	Deluge – form events (validate / success / field actions)	Y	Load of the form, Successful form submission, Validations on form submission	5.5, 5.6	Figures 5.11 and 5.12; Appendix C
II	Deluge – functions, fetch, loops, collections	Y	Fetch with criteria and for each loop in Recalculate Waiting Time; standalone function.	5.5	Code 5.1; Section C.1
II	E-mail / in-app notifications	Y	Two e-mails and Cliq channel messages	5.5, 5.6	Figures 5.14, 5.16 and 5.17
II	Form workflow	Y	Eight on Token and one on Service Counter	5.6	Figures 5.11 and 5.12
II	Scheduled workflow	N	Not evidenced	5.6	—

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Approval process	N	Not used	5.6	—
II	Blueprint	N	Not used	5.6	—
II	Batch / report workflow	N	Not used	5.6	—
II	Testing and debugging evidence	Y	info statement in Customer Assigned; test table and observations	6	Tables 6.1 and 6.3; Figures 6.1 and 6.2
III	Integration with a Zoho service	Y	Zoho Cliq through zoho_cliq_connection	5.7	Figures 5.13 and 5.14
III	Connection / REST API call (invokeurl)	Y	Connection used by the Cliq task; no invokeurl call evidenced	5.7	Figure 5.13
III	Webhook or Creator REST API	N	Not used	5.7	—
III	Users, roles and permission profiles	Y	2 roles, 3 profiles, 4 data-sharing rules, 3 users	5.8, 7.3	Figures 5.18 to 5.21
III	Customer portal	Y	Portal published with sign-in and Sign Up	5.11, 7.1	Figure 5.22
III	Record-level access criteria	Partial	Described in the Customer profile	5.8, 7.3	Figure 5.19
III	Zia / AI feature	N	Not Used	5.9	—
III	Analytics (charts / Zoho Analytics)	Y	Native dashboard charts; Zoho Analytics not used	5.9	Figure 5.10
IV	Performance optimisation applied and measured	N	Proposals only	7.2	Code 7.1
IV	Deployment across environments	Y	Published to Production (version 2.0); Stage not evidenced	7.1	Figure 5.1
IV	Security controls (encryption, OAuth, permissions)	Y	Profiles, data sharing, connection, closed channel; encryption not used	7.3	Figures 5.13, 5.15 and 5.18 to 5.20
IV	Governance (audit trail, administration)	Partial	Users page and release e-mail; audit trail not evidenced	7.4	Figure 5.21
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Not used	5.10	—
V	Low-code limitations and spectrum analysis	Y	Discussed	8.4, 8.5	—
—	Mobile app / portal access	Y	Portal published; mobile app not configured	5.11	Figure 5.22

Total features fully implemented: 23 / 33 (a further 2 are partially implemented).

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	[ ]	[ ]
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	[ ]	[ ]
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9, 7.3	[ ]	[ ]
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	[ ]	[ ]
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	[ ]	[ ]

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Recalculate Waiting Time (Service Counter → Edited → Successful form submission)

```
// Script ID : DS-09
// Location  : Service Counter form -> Edited -> Successful form submission
//             -> Recalculate Waiting Time
// Purpose   : Set the estimated waiting time of every waiting token of the
//             edited counter to (average service time x position in the queue)
if(input.average_service_time != null && input.average_service_time > 0)
{
    waitingTokens = token[service_counter == input.ID && token_status ==
"Waiting"];
    position = 1;
    for each tokenRecord in waitingTokens
    {
        tokenRecord.estimated_waiting_time=input.average_service_time * position;
        position = position + 1;
    }
}
```

C.2 Customer Assigned (Token → Edited → Successful form submission)

```
// Script ID : DS-07
// Location  : Token form -> Edited -> Successful form submission
//             -> Customer Assigned (the first of its two actions is shown)
// Purpose   : Post a message to the Zoho Cliq channel when a customer is
//             assigned to a service counter
message = "A customer has been assigned to a service counter.\n" + "Token Number:
" +
    ifnull(input.Token_Number,"").toString() + "\n" + "Customer Name: " +
    ifnull(input.customer_name,"").toString() + "\n" + "Service Counter: " +
    ifnull(input.service_counter,"").toString();
response =
zoho.cliq.postToChannel("smartqnotifications",message,"zoho_cliq_connection");
info response;
```

Seven further workflows are listed in the application but their script bodies were not captured. Paste each script from the Deluge editor in the place marked.

C.3 Calculate Estimated Waiting Time (Token → Created → Successful form submission (1 action))

```
if(input.service_counter != null)
{
    counterRecord = Service_Counter[ID == input.service_counter];

    if(counterRecord.ID != null && counterRecord.average_service_time != null)
    {
        waitingTokens = Token[service_counter == input.service_counter &&
token_status == "Waiting"];

        queuePosition = waitingTokens.count();

        input.estimated_waiting_time = queuePosition *
counterRecord.average_service_time;
    }
}
```

C.4 New_Token_Generated (Token → Created → Successful form submission (2 actions))

```
message = "New token generated successfully.\n"
+ "Token Number: " + ifnull(input.Token_Number,"").toString() + "\n"
+ "Customer Name: " + ifnull(input.customer_name,"").toString() + "\n"
+ "Service Counter: " + ifnull(input.service_counter,"").toString() + "\n"
+ "Issued At: " + ifnull(input.issued_at,"").toString() + "\n"
+ "Estimated Waiting Time: " +
ifnull(input.estimated_waiting_time,"").toString();

response = zoho.cliq.postToChannel("smartqnotifications", message,
"zoho_cliq_connection");

info response;
```

C.5 Notify Customer of Wait Time (Token → Created → Successful form submission (1 action))

```
message = "Your token has been generated successfully.\n"
+ "Token Number: " + ifnull(input.Token_Number,"").toString() + "\n"
+ "Estimated Waiting Time: " +
ifnull(input.estimated_waiting_time,"").toString();

response = zoho.cliq.postToChannel("smartqnotifications", message,
"zoho_cliq_connection");
info response;
```

C.6 Update Token Status (Token → Edited → Successful form submission (1 action))

```
if(input.token_status == "Served")
{
    info "Token " + input.Token_Number + " has been served successfully.";
}
else
{
    info "Token status updated: " + input.token_status;
}
```

C.7 Validate Queue Position (Token → Created or Edited → Validations on form submission (1 action))

```
if(input.service_counter != null)
{
    waitingTokens = Token[service_counter == input.service_counter &&
token_status == "Waiting"];

    if(waitingTokens.count() > 50)
    {
        alert "Maximum queue limit reached for this service counter.";
        cancel submit;
    }
}
```

C.8 Auto-Set Token Status (Token → Created → Load of the form (1 action))

```
input.token_status = "Waiting";
```

C.9 Token Number Fill…. (Token → Created → Load of the form (1 action))

```
existingTokens = Token[ID != 0].count();

nextNumber = existingTokens + 1;

if(nextNumber < 10)
{
    input.Token_Number = "SQ-000" + nextNumber.toString();
}
else if(nextNumber < 100)
{
    input.Token_Number = "SQ-00" + nextNumber.toString();
}
else if(nextNumber < 1000)
{
    input.Token_Number = "SQ-0" + nextNumber.toString();
}
else
{
    input.Token_Number = "SQ-" + nextNumber.toString();
}
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Open the application portal at https://smartq-qms.zohocreatorportal.in. Staff and administrators sign in with the Zoho account that has been assigned to their role. Customers enter their e-mail address on the sign-in page and choose Next, or choose Sign Up to create an account. The menu on the left shows Dashboard, Feedback, Service Counters, Token and App Preferences. The application can be opened in a desktop, tablet or phone browser; the Zoho Creator mobile app has not been configured.

D.2 Instructions for the Customer

1. Sign in to the portal (or choose Sign Up the first time).
2. Open Token → Token. The token number and the issue time are already filled.
3. Enter your name and e-mail address (both are mandatory) and, if you wish, your phone number and address.
4. Choose the service counter in the Service Counter list and select Submit.
5. Read the e-mail "Your Token – Estimated Wait Time" for your waiting time.
6. After the service, open Feedback → Feedback, choose your token number, give a rating, add comments and submit.

D.3 Instructions for Service Counter Staff

1. Sign in and open Token → Token Report (or Statuses) to see the tokens of your counter.
2. To call a customer, open the token record, choose Edit, change Token Status to Serving and submit.
3. After the service, change the status to Served, or to No-show if the customer does not respond.
4. Watch the channel #smartq-notifications in Zoho Cliq for new-token messages.
5. Staff cannot delete records or change counter settings.

D.4 Instructions for the Administrator

1. Open Service Counters → Service Counter. Enter the Counter Name, Service Type, Counter Status and Average Service Time (mandatory), the maximum capacity, the location and operating hours (use Add New for each day) and the manager's details, then submit.
2. Open Service Counters → Service Types to see the counters grouped by service type.
3. Open Dashboard to see the token-status tiles, the feedback tiles and the charts, and use the quick links to add a feedback record, a service counter or a token.
4. Open Feedback → All Feedback or Feedback Board to review feedback and its resolution status.
5. Open Settings → User Permissions to review the roles, the permission profiles and the data-sharing rules.
6. Edit the average service time of a counter to recalculate the waiting times of its waiting tokens.